LECTURE NOTES · THE TRANSFORMER, VISUALLY, PART TWO

Frontier Pre-training, Visually

Companion notes to the slides

How a frontier base model is built, following Microsoft AI's report on MAI-Base-2: scaling ladders and source scaling, hyperparameter laws, mixture-of-experts systems, the Muon optimizer at 16k GPUs, MXFP8 numerics and gradient clipping, and a 50T-token mixture with native vision. One section per slide, with every term defined and the intuition spelled out, and a reading list for each part.

For advanced undergraduates and beginning graduate students

October 2026

These notes follow the deck *Frontier Pre-training, Visually* (35 slides) slide by slide. The deck follows the Microsoft AI report *Frontier Pre-training at Microsoft AI* (microsoft.ai/news, 15 October 2026); "the report" always means that document, and "Figure *n*" always means its Figure *n*. Numbers quoted from the report are introduced with "the report says"; numbers marked "derived" were computed by the deck or these notes from the report's figures; numbers marked "read off" were read from a plot and are approximate; comparator facts come from the models' Hugging Face model cards and are marked as such. Curves the slides label "illustrative" are sketches of a mechanism, not data.

Contents

How to use these notes **3**

1 Overview (slides 1–3) **3**
1.1 Slide 1: Title: Frontier Pre-training, Visually . . . 3
1.2 Slide 2: Where Part One ended, and what this deck adds . . . 4
1.3 Slide 3: MAI-Base-2 in numbers . . . 5

2 Results (slides 4–5b) **6**
2.1 Slide 4: Reading the metric: bits per byte . . . 6
2.2 Slide 5: Figure 1: held-out bits per byte, open models above 1T . . . 8
2.3 Slide 5b: The comparators, from their model cards . . . 9

3 The model (slides 6–8) **10**
3.1 Slide 6: Two models side by side . . . 10
3.2 Slide 7: Background: the MLP becomes a mixture of experts . . . 12
3.3 Slide 8: Background: attention and embedding choices . . . 13

4 The three initiatives (slide 9) **14**
4.1 Slide 9: Three initiatives, three sections of this deck . . . 14

5 Scaling ladders (slides 10–14) **15**
5.1 Slide 10: Scaling ladders: one number sets the whole model . . . 15
5.2 Slide 11: Background: scaling laws and tokens per parameter . . . 16
5.3 Slide 12: Source scaling: repeat the data alike on every rung . . . 17
5.4 Slide 13: Figure 2: the ladder forecast the final model . . . 18
5.5 Slide 14: Figure 3: how much compute is the recipe worth? . . . 20

6 Hyperparameter laws (slides 15–18) **21**
6.1 Slide 15: LR and weight decay: one knob and a half . . . 21
6.2 Slide 16: Figure 4, top: fit the loss surface, do not pick the best run . . . 22
6.3 Slide 17: Figure 4, bottom: the laws behind the hyperparameters . . . 23
6.4 Slide 18: Figure 5: a run with no spikes . . . 24

7 MoE systems (slides 19–23) **26**
7.1 Slide 19: Co-design: score every idea in FLOPs and in wall-clock time . . . 26
7.2 Slide 20: Scaling MoE: two inefficiencies, then MoE in every layer . . . 27
7.3 Slide 21: Background: the capacity problem in expert parallelism . . . 28
7.4 Slide 22: Figure 6: multi-round dropless MoE, one forward pass . . . 30
7.5 Slide 23: Cutting per-expert overhead: four changes . . . 31

8 Muon (slides 24–25) **32**
8.1 Slide 24: Background: Muon orthogonalizes the update . . . 32
8.2 Slide 25: Muon on 16k GPUs: reshard, orthogonalize, reshard back . . . 34

9 Numerics and stability (slides 26–29) 36
9.1 Slide 26: Numerics: from tensor-wise FP8 to MXFP8 . . . 36
9.2 Slide 27: Background: global versus activation gradient clipping . . . 37
9.3 Slide 28: Figure 7: gradient norm, per step and rolling median . . . 38
9.4 Slide 29: Figures 8 and 9: a gain that grows with scale, and a bug it found . . . 40

10 Data (slides 30–32) 41
10.1 Slide 30: Scaling data: every token processed in-house . . . 41
10.2 Slide 31: Figure 10: the mixture, 30T tokens to 50T . . . 43
10.3 Slide 32: Native vision without an image encoder . . . 44

11 Wrap-up (slides 33–34) 45
11.1 Slide 33: Part One's components, Part Two's choices . . . 45
11.2 Slide 34: Sources . . . 46

A Notation 48

B Formulas at a glance 49

C Exercises 51

D Reading list by section 54

Index of terms 59

How to use these notes

Each section of these notes corresponds to one section of the deck, and each subsection to one slide. A subsection has four parts:

- **What the slide shows**: a description of the figure or table, so the notes can be read without the slides.
- **Details**: the material the slide compresses, with the equations written out and the numbers given their sources.
- A **Terms** box defining every technical term the slide uses, the first time it appears. All terms are collected in the index at the end.
- An **Intuition** box: the picture to keep in mind, and the question to ask when the picture seems to fail.

Conventions. N_{total} is the number of parameters a model stores and N_{active} the number it uses for one token; for a dense model they are equal. L is the number of layers of a model (the report names each rung of its scaling ladder by L); where a loss is meant it is written $\mathscr{L}$. D is a number of training tokens and C a training compute in FLOPs; FLOP/s is a rate; FLOPs is a count. "T" and "B" after a number mean 10^{12} and 10^{9}. BPB is bits per byte of held-out text, lower is better. log without a base is the natural logarithm, which is also what the report's hyperparameter laws use (Section 6). The report's two models are written Base-1 and Base-2 for short.

A reading plan for a single lecture: Sections 1–3 (what the model is and how it scores) take the first quarter; Sections 5 and 6 (ladders and hyperparameter laws) are the scientific core and take a third; Sections 7–9 (MoE systems, Muon, numerics and clipping) are the systems half and take another third; Sections 10 and 11 close the loop with data and the three principles. Section 4 is a one-slide roadmap. Appendix D is the reading list, section by section, and matches the paper archive that accompanies these notes.

1 Overview (slides 1–3)

1.1 Slide 1: Title: Frontier Pre-training, Visually

How MAI-Base-2 was built: scaling ladders, hyperparameter laws, mixture-of-experts systems, Muon, stability and data. A visual companion to Microsoft AI's report Frontier Pre-training at Microsoft AI *(15 October 2026).*

What the slide shows The cover graphic is a scaling ladder drawn as a bar chart: five green bars of increasing height labelled L18, L24, L30, L36 and L42, a dashed line continuing their trend, and one much taller orange bar labelled L102 and "MAI-Base-2". The horizontal axis is training compute. The picture is the deck's thesis in one image: the big model is the top rung of a ladder of small models, and everything about it was decided on the lower rungs.

Details This deck is Part Two of *The Transformer, Visually*. Part One explained the transformer block: tokens, embeddings, attention, the MLP, the residual stream, the output softmax and the loss. Part Two is about the *recipe*: how a frontier laboratory decides what model to build, how it sets the training hyperparameters, how it makes the training run fast on sixteen thousand GPUs, how it keeps the run stable for weeks, and what it feeds the model. The vehicle is a single document, the report Microsoft AI published on 15 October 2026 about MAI-Base-2, a mixture-of-experts base model with 80 billion active and 2.8 trillion total parameters. The report is thirteen pages long and has ten figures; the deck reproduces all ten and reads each one closely.

The deck has ten footer stages: Overview, Results, Model, Ladders, Hyperparameters, MoE systems, Muon, Stability, Data, Wrap-up. These notes have one section per stage, plus a short Section 4 for the

roadmap slide. Two kinds of slide alternate throughout: slides that read a report figure or table, and slides marked "Background" that supply what the report assumes the reader knows (what a mixture of experts is, what Muon does, what FP8 scaling is). The report is written for practitioners; the background slides are what make it readable by students who have only seen Part One.

Terms

pre-training The first and by far the largest stage of training a language model: next-token prediction on a very large corpus, which gives a *base model*. Post-training (instruction tuning, reinforcement learning) comes afterwards and is a different deck.

base model The model as it comes out of pre-training, before any post-training. MAI-Base-2 is a base model; the report compares it with other models' base checkpoints where those exist.

frontier model A model at or near the largest scale anyone has trained at the time. The report defines its comparison set as open-weight models with at least one trillion total parameters.

scaling ladder A family of models of different sizes trained with one fixed recipe, used to test changes cheaply and to extrapolate to the full-size model. Section 5.

mixture of experts (MoE) A layer made of many parallel MLPs (experts) and a router that sends each token to a few of them. The parameters of unused experts are stored but not touched, so the model's total parameter count can far exceed the number used per token.

Intuition

Part One was anatomy: what the organs are and how they connect. Part Two is physiology and medicine: how the organism is grown, kept healthy, and fed. The ladder on the cover is the central instrument. A frontier lab cannot afford to try things on the big model, so it tries them on small models arranged so that what works on them will work at scale, and the main scientific content of the report is how to arrange them so.

1.2 Slide 2: Where Part One ended, and what this deck adds

Part One explained the block. Part Two is about the recipe: how a frontier lab decides what to build and how to train it.

What the slide shows The top half is the transformer block from Part One: Norm, Attention, a residual addition, Norm, MLP, a residual addition, with the note "one block, repeated L times". Under the Attention box a callout says 96 query heads, grouped-query, local and global layers; under the MLP box, "becomes a mixture of 512 experts, 8 active per token"; to the right, "stacked 102 times: 2.8T parameters in total, 80B active per token". The bottom row asks the three questions the deck answers, in the colors of the three sections that answer them: *How do you choose?* (ladders and hyperparameter laws), *How do you make it fast?* (MoE kernels, Muon, clipping), *What do you feed it?* (50 trillion tokens of text and images).

Details Everything in the callouts is from the report's architecture table (slide 6). The block is the same block as in Part One; what changes is the size of each piece and the fact that the MLP is replaced by a mixture of experts. "8 active per token" counts one shared expert plus seven routed ones (slide 7). "2.8T total, 80B active" is the pair of numbers that characterizes a mixture-of-experts model: $N_{\text{total}} = 2.8 \times 10^{12}$ parameters are stored, $N_{\text{active}} = 8 \times 10^{10}$ are used for any one token, a ratio of 35.

The three questions map onto the report's three "initiatives" (slide 9): scaling science, model–infrastructure co-design, and scaling data. The deck follows the report's order, which is also the logical order: decide what to build (Sections 5–6 of these notes), build the systems that make it affordable (Sections 7–9),

and feed it (Section 10).

Terms

active parameters (N_{active}) The parameters that participate in computing one token's output. In a dense model this is every parameter; in an MoE it is the attention, the shared expert, the router and the k routed experts chosen for that token.

total parameters (N_{total}) Every parameter the model stores, including all experts. Sets memory, not per-token compute.

grouped-query attention (GQA) Several query heads share one key/value head, which shrinks the KV cache. Slide 8.

local and global layers Attention layers that attend only within a window (local) versus over the whole context (global). Slide 8.

recipe Everything about a training run other than the data: architecture, parameterization, optimizer, hyperparameters, numerics, schedule. The ladder holds the recipe fixed and varies only the size.

Intuition

If Part One's block is a Lego brick, Part Two is about the factory: how big to make the bricks, how many to stack, which machines press them fast enough, and where the plastic comes from. None of the three questions can be answered by looking at one brick.

1.3 Slide 3: MAI-Base-2 in numbers

A mixture-of-experts model trained on 16k GB200 GPUs over 38 days in August and September 2026.

What the slide shows Eight large numbers in two rows, each with a one-line caption: 80B / 2.8T (active / total parameters); 102 (layers, hidden width 8,704); 50T (training tokens, text and vision); 16k (GB200 GPUs, 38 days); 21% (MFU, against a 2.5 PFLOPS BF16 peak); 89% (goodput); $\approx 2.4 \times 10^{25}$ (training FLOPs, derived); 4–12× (FLOPs-efficiency over the best open base model). Under them, a background panel checks that the numbers agree with each other.

Details The report says: 80 billion active and 2.8 trillion total parameters; 102 layers; hidden dimension 8,704; a 50T-token training horizon; trained on 16k GB200 GPUs over 38 days in August and September 2026; average MFU 21%, normalized by 2.5 PFLOPS, the GB200's theoretical BF16 maximum, and excluding FLOPs for rematerialization and the optimizer; goodput, the fraction of time in which training progressed, 89%; and 4–12× more pre-training FLOPs-efficient than DeepSeek-V4-Pro-Base, the strongest open base model (Figure 3, slide 14).

The training FLOPs are the deck's derivation, not a report number. Part One's cost model gives $C \approx 6N_{\text{active}}D$ for a dense transformer, 2 FLOPs per parameter per token forward and 4 backward; for an MoE the same count holds with N_{active} in place of N, because the unused experts do no arithmetic. Hence

$$C \approx 6 \times (80 \times 10^9) \times (50 \times 10^{12}) = 2.4 \times 10^{25} \text{ FLOPs (derived).}$$

The slide checks this against the hardware. Sixteen thousand GPUs at 2.5×10^{15} FLOP/s each, for 38 days of 86,400 s, is 1.3×10^{26} FLOPs of peak capacity; multiplying by the 21% MFU and the 89% goodput gives 2.45×10^{25} (derived). The two estimates agree to within the rounding of the inputs, and both agree with the position of the last point on the report's Figure 2, which sits at roughly $10^{25.4} \approx 2.5 \times 10^{25}$ FLOPs (read off the plot). The agreement is not a coincidence but it is a useful check: the report's MFU definition

excludes rematerialization and optimizer FLOPs, which is exactly what the $6ND$ count also excludes, so the two sides are measuring the same thing.

Two of the eight numbers deserve a second look. MFU of 21% is low by the standards of dense models (PaLM reported 46% on TPU v4; Llama 3 reported 38–43% on H100s), and the MoE systems section explains why: a fine-grained MoE with 512 small experts does many small matrix multiplications wrapped in communication, and the report says its MoE layers ran at 24% MFU before its systems work and 35% after (slide 20). Goodput of 89% means that eleven percent of the 38 days, about four days, went to restarts, stragglers, checkpointing and other time in which the step counter did not advance.

Terms

FLOPs Floating-point operations, a count; one multiply-add is two FLOPs. FLOP/s (or FLOPS) is the rate at which a device performs them. The report writes PFLOPS for 10^{15} FLOP/s.

model FLOPs utilization (MFU) The fraction of a device's peak arithmetic rate that the training step actually uses, counting only the forward and backward model FLOPs (PaLM, 2022). Rematerialization, optimizer steps and communication count as overhead, not as useful FLOPs.

goodput The fraction of wall-clock time in which training made progress. The complement is time lost to failures, restarts, stragglers and checkpoints.

rematerialization Recomputing activations in the backward pass instead of storing them from the forward pass; trades compute for memory. Also called activation recomputation or checkpointing. Slide 20.

GB200 NVIDIA's Blackwell-generation GPU as deployed in the GB200 NVL72 rack; the report takes its BF16 dense peak as 2.5 PFLOPS. The native support for microscaling formats (slide 26) is a Blackwell feature.

the $6ND$ rule Training compute $\approx 6N_{\text{active}}D$: 6 FLOPs per active parameter per training token (Kaplan et al. 2020). Appendix B.

Intuition

Every big number on the slide can be derived from two others, and that is the point of the slide: a frontier training run is an exercise in bookkeeping. Parameters times tokens gives FLOPs; FLOPs divided by what the hardware can do gives days; the two percentages are the taxes that reality levies on the plan. If a reported run's numbers do not close like this, something is being counted differently, and the first question is which definition of MFU is in use.

2 Results (slides 4–5b)

2.1 Slide 4: Reading the metric: bits per byte

Every comparison in the report is held-out bits per byte: a compression score that does not depend on the tokenizer.

What the slide shows A line of code, `def area(r):  return pi*r**2`, labelled "held-out text, B bytes of UTF-8", is cut into eight token chips. Above each chip is the probability the model assigned to that token given everything before it; below each chip is a bar whose height is $-\log_2 p$, the number of bits the token costs. Probabilities and bars are illustrative. Under the chips is the definition,

$$\mathrm{BPB} = \frac{1}{B}\sum_{i=1}^{N} -\log_2 p(x_i \mid x_{<i}) = \frac{N}{B}\cdot\frac{\overline{\mathrm{NLL}}}{\ln 2},$$

with the remark that the tokens cancel. The right panel gives three reasons the report uses this metric: it is a compressor's score; the evaluation sets are held out on purpose; and three of the comparators are post-trained checkpoints, a caveat the report keeps.

Details A language model assigns each token x_i a probability $p(x_i \mid x_{<i})$ given its prefix. The negative log-likelihood of a text is $\sum_i -\ln p(x_i \mid x_{<i})$ nats; divided by the number of tokens N it is the mean NLL per token, $\overline{\mathrm{NLL}}$, the quantity training minimizes. Converting nats to bits divides by $\ln 2$, and dividing by the number of bytes B in the original text instead of the number of tokens gives bits per byte. The identity on the slide is just this chain of conversions; the factor N/B is the tokenizer's tokens per byte.

Why bytes rather than tokens. Two models with different tokenizers see different token sequences for the same text, so their per-token losses are not comparable: a tokenizer with a larger vocabulary cuts the text into fewer, longer tokens, each of which is harder to predict. Per byte of the original text the two are comparable, because the thing being predicted is the same text. BPB is therefore the standard metric for comparing models across tokenizers; The Pile (Gao et al. 2020) popularized it, and the report says it uses it because it is "agnostic to each model's tokenizer".

The compression reading. A model with a probability for every next symbol can drive an arithmetic coder, which encodes a text in a number of bits equal to its total $-\log_2 p$ plus a constant (Delétang et al. 2023 work this out for language models). So a BPB of 0.22 on code means the model, used as a compressor, would shrink the code to 0.22 bits per byte, a factor $8/0.22 \approx 36$ (derived). Lower is better, and the differences in Figure 1 are differences in compression ratio.

What is held out. The report says held-out code is MAI's own codebase; the STEM, Math and Q&A sets were commissioned from vendors to reduce the risk of contamination; and all evaluation sets were decontaminated against the training data. It also says that only post-trained checkpoints are available for Kimi K3, Qwen3.8 and MiMo-V2.6-Pro-RL, and that post-training may affect these evaluations. Post-training usually raises the loss on raw text somewhat, because the model has been pulled toward a chat distribution, so the caveat cuts in the comparators' favor: their base checkpoints would likely score better than the starred numbers.

Terms

bits per byte (BPB) Total bits a model needs to encode a text, $\sum_i -\log_2 p(x_i \mid x_{<i})$, divided by the text's size in bytes. Tokenizer-independent; lower is better.

negative log-likelihood (NLL) $-\ln p$ of the observed token; the training loss, in nats per token when averaged.

nat, bit Units of information: $-\ln p$ is in nats, $-\log_2 p$ in bits; one nat is $1/\ln 2 \approx 1.44$ bits.

arithmetic coding A lossless compressor that, given a probability model, encodes a sequence in about $\sum -\log_2 p$ bits. Any language model is implicitly a compressor.

held-out evaluation A test set kept out of training and, here, actively removed from the training data (decontamination) so that low loss cannot come from having seen the text.

decontamination Searching the training corpus for overlaps with the evaluation sets and removing them.

post-trained checkpoint A model after instruction tuning or reinforcement learning. Its raw-text loss is not that of its base model.

Intuition

Think of each model as a compressor and of the held-out sets as the files to be compressed. The best model is the one that produces the smallest file, and "bits per byte" is just the compressed size divided by the original size. Tokenizers are the compressor's internal choice of how to chunk the input, which the file size does not care about. The question to ask of any BPB number is what text it was measured on, which is why the report spends a paragraph on where its evaluation sets came from.

2.2 Slide 5: Figure 1: held-out bits per byte, open models above 1T

Lower is better. Green is MAI-Base-1, orange is MAI-Base-2; starred comparators are post-trained checkpoints.

What the slide shows The report's Figure 1, reproduced at the left, and next to it a table of its values. The figure has four panels: Held-Out Code (MAI's codebase), Held-Out QA, Held-Out STEM and Held-Out Math. In each, the vertical axis is bits per byte, lower is better, with the axis range chosen to spread the bars (0.215–0.250 for code, 0.41–0.46 for QA, 0.35–0.41 for STEM, 0.35–0.40 for math), and six bars stand for, left to right, MAI-Base-1 (35B/1T), MiMo-V2.6-Pro-RL* (42B/1.02T), Qwen3.8-2.4T-A95B* (95B/2.4T), DeepSeek-V4-Pro-Base (49B/1.6T), Kimi K3* (104B/2.8T) and MAI-Base-2 (80B/2.8T). Each bar is labelled with its value to four decimals. The orange MAI-Base-2 bar is the lowest in every panel.

Details The values, read from the figure's labels (the report does not tabulate them):

Model (active / total)	Code	QA	STEM	Math
MAI-Base-1 (35B / 1T)	.2401	.4440	.4026	.3939
MiMo-V2.6-Pro-RL* (42B / 1.02T)	.2446	.4513	.3975	.3803
Qwen3.8-2.4T-A95B* (95B / 2.4T)	.2400	.4427	.3978	.3752
DeepSeek-V4-Pro-Base (49B / 1.6T)	.2370	.4351	.3968	.3862
Kimi K3* (104B / 2.8T)	.2264	.4269	.3761	.3589
MAI-Base-2 (80B / 2.8T)	**.2203**	**.4162**	**.3653**	**.3561**

The report says that MAI-Base-2 achieves lower BPB than every other model it tested, and that the comparison set is frontier checkpoints available before October 2026 with at least one trillion total parameters: DeepSeek-V4-Pro-Base as the most capable open base model, and Kimi K3, Qwen3.8 and MiMo-V2.6-Pro-RL as post-trained open-weight models (the asterisks).

The margins are the deck's derivation from the table. Over the runner-up, Kimi K3: $(0.2264 - 0.2203)/0.2264 = 2.7\%$ on code, 2.5% on QA, 2.9% on STEM and 0.8% on math. Over MAI-Base-1: 8.2% on code, 6.3% on QA, 9.3% on STEM and 9.6% on math, which the slide rounds to 8, 6, 9 and 10%. The predecessor comparison comes with $80/35 = 2.3\times$ the active parameters and $50/30 = 1.7\times$ the training tokens, so it mixes recipe improvements with scale; Figure 3 (slide 14) is the report's attempt to separate the two.

How to read the sizes. The four comparators span 1.02T to 2.8T total and 42B to 104B active parameters. Kimi K3 has the same total as MAI-Base-2 and 30% more active parameters; DeepSeek-V4-Pro-Base has 60% of the active parameters. Among models of similar size the ordering tracks neither total nor active parameters alone, which is one reason the report insists on normalizing by compute.

A reading note on small differences. A 0.8% margin on math is small; BPB differences of this size can come from how the held-out set was drawn, and the report gives no error bars. The code, QA and STEM margins are three times larger and consistent across panels, which is the more robust fact.

Terms

comparator A model the report measures against its own. The report's criterion is open weights and at least 1T total parameters.

open-weight model A model whose weights are published for download, as opposed to one served only through an API. Open weights do not imply open data or code.

runner-up margin The relative difference between the best and second-best score; the deck's measure of how far ahead MAI-Base-2 is.

Intuition

Four races, same six runners, same winner each time. The interesting question is not who won but by how much and at what cost, and "at what cost" is what the rest of the Results and Ladders sections is about. Keep the two caveats in mind: three runners were post-trained (which probably slowed them on this test), and nobody reported error bars.

2.3 Slide 5b: The comparators, from their model cards

What the five models in Figure 1 look like inside, per their Hugging Face cards and configs (October 2026).

What the slide shows A six-row table, one row per model in Figure 1, with columns for total/active parameters, layers, experts (routed / active / shared), attention layout, and whether base weights are released. Below the table are three readings: convergence (all six are fine-grained MoEs mixing a few global-attention layers with local, linear or compressed attention); where MAI differs (wide experts, few per token, plain softmax attention throughout, the sparsest global ratio); and compute context (a lower bound on DeepSeek-V4-Pro-Base's training FLOPs). The footnote says the MAI rows come from the report and the others from Hugging Face model cards and `config.json` files, and that MiMo's global/local reading is inferred from its config.

Details Everything in the non-MAI rows is from the comparators' model cards, not from the report, and should be read as such; model cards are secondary sources and can change. As the deck transcribes them:

- MAI-Base-1 (report): 1T / 35B, 78 layers, 512 routed experts with 8 active and no shared expert, GQA with a global layer every sixth layer.
- MiMo-V2.6-Pro-RL (model card): 1.02T / 42B, 70 layers, 384 routed with 8 active, 60 sliding-window layers (window 128) plus 10 global, 128 query and 8 KV heads.
- Qwen3.8-2.4T-A95B (model card): 2.4T / 95B, 92 layers, 512 routed with 10 active and 1 shared, 69 Gated DeltaNet layers plus 23 attention layers (one in four), 64 query and 4 KV heads.
- DeepSeek-V4-Pro-Base (model card): 1.6T / 49B, 61 layers, expert counts not stated on the card, "compressed sparse" and "heavily compressed" attention alternating; base weights released, trained on more than 32T tokens.
- Kimi K3 (model card): 2.8T / 104B, 93 layers, 896 routed with 16 active and 2 shared, 69 Kimi Delta Attention layers plus 24 gated MLA layers (one in four); weights shipped in MXFP4.
- MAI-Base-2 (report): 2.8T / 80B, 102 layers, 512 routed with 7 active and 1 shared, 85 local plus 17 global attention layers (one in six), 96 query heads with 2 KV heads in global and 16 in local layers; weights not released at the time of the report.

The convergence the slide points to is real: every model on the list uses hundreds of small experts with 7–16 active, and every one mixes a minority of full-context attention layers with something cheaper for the rest,

whether a sliding window (MiMo, MAI), a linear-attention recurrence (Gated DeltaNet in Qwen3.8, Kimi Delta Attention in Kimi K3) or compressed attention (DeepSeek-V4). MAI's choices sit at the plain end of this space: ordinary softmax attention in every layer, local or global, and experts that are wide (8,704) rather than numerous.

The compute context is the deck's derivation. DeepSeek's card says more than 32T tokens at 49B active, so by the $6ND$ rule its training compute is at least $6 \times 49 \times 10^9 \times 32 \times 10^{12} \approx 9.4 \times 10^{24}$ FLOPs; MAI-Base-2's 2.4×10^{25} is at most about $2.5\times$ that. Figure 3 (slide 14) places DeepSeek-V4-Pro-Base at about 10^{25} FLOPs (read off the plot), consistent with this bound. The point is that the 4–12$\times$ efficiency claim of Figure 3 is not explained by MAI-Base-2 having spent far more compute; it spent roughly twice as much.

Terms

model card The documentation page that accompanies published weights (on Hugging Face, a README plus a `config.json`); the deck's source for every non-MAI architecture fact.

fine-grained experts Many small experts rather than a few large ones (DeepSeekMoE, 2024); all six models here have hundreds.

shared expert An expert that every token uses, in addition to the routed ones; carries what all tokens need so the routed experts can specialize.

sliding-window attention Attention restricted to the last w tokens; cost and cache are linear in context length. MiMo's window is 128 tokens per its card.

linear attention, Gated DeltaNet, Kimi Delta Attention Recurrent layers that replace the softmax attention of some layers with a fixed-size state updated token by token; cost linear in context. Gated DeltaNet is Yang et al. 2024; Kimi Delta Attention is from Kimi Linear (2025).

multi-head latent attention (MLA) DeepSeek's attention variant that caches a compressed latent per token instead of full keys and values.

MXFP4 A 4-bit microscaling float format (blocks of 32 values sharing an 8-bit scale), used by Kimi K3 for its released weights; slide 26 covers the 8-bit sibling.

Intuition

Put six frontier models side by side and the architecture looks almost settled: hundreds of experts, a few active, most attention layers cheap and a few global. What differs is the choice of cheap layer and the width of the experts. The report's view, visible in this table, is that plain components scaled with care beat clever ones: there is no linear attention and no latent compression in MAI-Base-2, only the oldest trick in the book, a sliding window, applied to five layers in six.

3 The model (slides 6–8)

3.1 Slide 6: Two models side by side

MAI-Base-2 is larger, sparser and trained longer than MAI-Base-1, and almost every line of the recipe changed.

What the slide shows The report's architecture table, transcribed: thirteen rows comparing MAI-Base-1 and MAI-Base-2. To its right, a sketch of the layer layout (Base-1 alternates dense-FFN and MoE layers; Base-2 has an MoE in every layer) and two readings: Base-2 is sparser, with $80/2800 = 2.9\%$ of its weights

acting on each token against 3.5% for Base-1; and MoE in every layer always won on FLOPs, and faster kernels made it win on wall-clock time too.

Details The report's table, row by row (Base-1 → Base-2):

- Active / total parameters: 35B / 1T → 80B / 2.8T.
- Layers: 78 → 102. Hidden dimension: 6,656 → 8,704.
- MoE layout: alternating dense FFNs and MoEs → MoE in every layer. Dense FFN dimension: 13,312 → not applicable.
- MoE down-projection dimension: 3,072 → 4,096. MoE expert FFN dimension: 10,240 → 8,704.
- Routing / expert count: 0 shared + 8 of 512 routed → 1 shared + 7 of 512 routed; the report adds that the shared expert is not down/up projected.
- Attention heads (KV / Q): 8 / 80 in both global (every sixth layer) and local layers → 2 / 96 in global layers, 16 / 96 in local layers.
- Embeddings: tied → untied.
- Training horizon: 30T → 50T tokens.
- Optimizer: AdamW → Muon for non-embedding 2-D parameters, AdamW for the rest.
- FP8 scheme: tensor-wise delayed scaling with E4M3 / E5M2 → MXFP8 with E4M3.
- Gradient clipping: global weight-gradient clipping → global weight-gradient plus activation-gradient clipping.
- Vision: text only → native VLM.

Several rows are explained by later slides: the MoE layout by slides 7 and 20–23, the attention heads by slide 8, the optimizer by slides 24–25, FP8 by slide 26, clipping by slides 27–29, vision by slide 32. The table is the agenda of the deck.

Three readings. First, the model got bigger in every dimension except the per-expert width: 31% more layers, 31% wider, 2.8× the total parameters, 2.3× the active parameters, 1.7× the tokens. Second, it got sparser: the active fraction $N_{\text{active}}/N_{\text{total}}$ fell from $35/1000 = 3.5\%$ to $80/2800 = 2.9\%$ (derived), partly because every layer is now an MoE and partly because the expert count per token went from 8 routed to 7 routed plus 1 shared. Third, the "MoE down projection" row is the one students ask about. The deck reads it as a projection of the 8,704-wide residual stream down to 4,096 before the routed experts, so that each routed expert works on a compressed copy of the stream, while the shared expert reads the full stream ("not down/up projected"). The report does not spell out the wiring beyond the two numbers, so this reading is an interpretation; the parameter arithmetic in Exercise 3 is consistent with it.

Terms

hidden dimension (d) The width of the residual stream, the vector each token carries between layers. 8,704 for Base-2.

expert FFN dimension The inner width of an expert's MLP, the size of the vector between its up and down projections. 8,704 for Base-2's routed experts.

MoE down projection A projection that reduces the stream's width before the routed experts; 4,096 in Base-2 per the report. The deck's reading is that routed experts see this compressed copy and the shared expert does not.

tied / untied embeddings Whether the input embedding table and the output (unembedding) matrix are the same parameters. Slide 8.

active fraction $N_{\text{active}}/N_{\text{total}}$; the share of stored weights used per token. 2.9% for Base-2.

MXFP8 Microscaling FP8: 8-bit floats in blocks of 32 that share one 8-bit scale. Slide 26.

Intuition

Read the table as a diff. Nearly every line changed, yet the report insists the second model was *predictable* from the first (Figure 2). The resolution is that none of the changes was made at full scale: each was tested on the ladder, and the big run only combined what had already been proven to transfer. The table is a list of conclusions; the ladder is the method.

3.2 Slide 7: Background: the MLP becomes a mixture of experts

Part One's MLP, replicated 512 times. A router sends each token to 7 of them plus one expert that everybody uses.

What the slide shows A token vector of width $d = 8{,}704$ enters a Router, which produces scores for 512 experts. A row of expert boxes follows, most faint and seven lit; at the end a separate "shared" expert marked "always on"; finally a Σ that combines the outputs, weighted by the router scores. Below: each token runs 8 expert MLPs of width 8,704 while all 512 are stored, which is how 2.8T parameters cost 80B per token; and the 4,096-wide down projection reads as routed experts seeing a compressed copy of the stream. A background panel traces the design to Shazeer et al. 2017, GShard and Switch Transformer (2020–21), and DeepSeekMoE (2024).

Details In Part One the MLP in each block is one feed-forward network applied to every token. A mixture-of-experts layer keeps the structure but has E such networks (the experts) and a small router, a linear map from the token vector to E scores. The k highest-scoring experts are run on the token and their outputs combined with weights from the scores:

$$g = \text{softmax}(W_r x) \text{ or a sigmoid gate;} \qquad y = x + \sum_{i \in \text{Top}k(g)} g_i E_i(x) + E_{\text{shared}}(x).$$

For MAI-Base-2 the report says $E = 512$ routed experts, $k = 7$, plus one shared expert that every token uses, so eight expert MLPs run per token. The parameters of the other 505 experts are stored (they count toward N_{total}) but do no arithmetic for this token (they do not count toward N_{active}). This is the entire mechanism by which 2.8T parameters cost 80B per token. The router's exact form (softmax or sigmoid, with or without a bias) is not stated in the report.

Where the design comes from. Shazeer et al. (2017) introduced the sparsely gated MoE layer: a learned router that makes a layer's cost independent of its parameter count, plus auxiliary losses that keep the experts evenly used (without them, the router collapses onto a few experts). GShard (2020) put MoE layers in a transformer at 600B parameters with top-2 routing and introduced expert parallelism and capacity factors (slide 21). Switch Transformer (2021) simplified to top-1 routing and a trillion parameters. DeepSeekMoE (2024) made the experts *fine-grained*, many and small, so the router has more combinations to choose from, and added *shared* experts that are always on, so the routed experts can specialize instead of each re-learning what every token needs. DeepSeek-V3 uses 1 shared + 8 of 256; MAI-Base-2 uses 1 shared + 7 of 512, the same idea with twice the routed pool.

The down projection. The deck's reading of the report's "MoE down projection dimension 4,096" is that the 8,704-wide stream is projected to 4,096 before the routed experts and back after, so each routed expert's

input and output are 4,096 wide while its inner width is 8,704; the shared expert, which the report says is not down/up projected, reads and writes the full 8,704. A routed expert then costs $\approx 2 \times 4{,}096 \times 8{,}704 \approx 71$M parameters if it has two matrices, or ≈ 107M with a separate gate; Exercise 3 shows that the only interpretation consistent with a 2.8T total is that the gate and up projections together span the 8,704 width.

Terms

router (gate) The small linear layer that scores every expert for a token; its top-k scores pick the experts and weight their outputs.

top-k routing Running only the k highest-scoring experts. $k = 7$ routed for Base-2.

routed vs shared expert A routed expert is chosen per token by the router; a shared expert is applied to every token unconditionally.

sparsely gated MoE Shazeer et al. 2017's name for the layer: the gate is sparse because most of its outputs are zero.

expert parallelism Placing different experts on different GPUs and shipping tokens to them; slide 21.

load balancing Keeping the number of tokens per expert roughly even, usually with an auxiliary loss (Switch's $\alpha \sum_i f_i P_i$) or a learned bias; slide 23 batches this loss across layers.

Intuition

A hospital with 512 specialists and one general practitioner. Every patient sees the GP and seven specialists chosen at the front desk; the other 505 specialists are on staff, drawing no cost for this patient, ready for the next one. The hospital's capability is the size of its staff; its cost per patient is eight consultations. The front desk is the router, and it must be good at its job and must not send everyone to the same seven doctors.

3.3 Slide 8: Background: attention and embedding choices

Five local layers for every global one, few key/value heads where the cache is unbounded, and untied embeddings.

What the slide shows A strip of 102 small boxes with every sixth lit, captioned "17 global, 85 local". Below, two head pictures: a global layer with 96 query heads and 2 K/V heads (48 queries per K/V head; the cache holds every token), and a local layer with 96 query heads and 16 K/V heads (6 per K/V head; the cache holds only the window). The reading: Base-1 used 8 K/V heads in both kinds of layer; Base-2 spends K/V capacity where caching is cheap (local, 16) and saves it where every token is cached (global, 2). The background panel covers grouped-query attention, local plus global layers, and untied embeddings.

Details The report says MAI-Base-2 has 96 query heads, 2 KV heads in global layers, which occur every sixth layer, and 16 KV heads in local layers. With 102 layers, every sixth gives $102/6 = 17$ global and 85 local layers (derived). The deck's ratios: $96/2 = 48$ query heads share each K/V head in a global layer, $96/16 = 6$ in a local layer. The report does not state the head dimension, the local window size, the positional encoding, or the context length.

Grouped-query attention (Ainslie et al. 2023). In multi-head attention every query head has its own key and value heads. In GQA the query heads are divided into groups, and each group shares one key head and one value head. The KV cache, which stores the keys and values of every past token so that generation does not recompute them, shrinks by the group size: a global layer of Base-2 stores keys and values for 2 heads rather than 96. Multi-query attention (one K/V head for all queries) is the limiting case.

Why the asymmetry between local and global layers. A local (sliding-window) layer attends only to the last w tokens, so its KV cache holds at most w entries however long the context; the cache is cheap, and the layer can afford 16 K/V heads. A global layer attends to the whole context, and its cache grows with it; at long contexts the global layers' caches dominate memory, so the design squeezes them to 2 K/V heads. Base-1 used 8 in both. Alternating local and global layers is itself a cost measure: local layers cost linear rather than quadratic time in the context length; the global layers carry long-range information. Gemma 2 alternates one to one; MAI-Base-2 uses five to one, the sparsest ratio in the comparator table of slide 5b.

Untied embeddings. The input embedding table (token id $\rightarrow$ vector) and the output unembedding (vector $\rightarrow$ logits over the vocabulary) have the same shape, $V \times d$, and can be the same matrix. Tying them (Press and Wolf 2017) saves $V \times d$ parameters and acts as a regularizer; it was standard in small models and Base-1 used it. At 2.8T parameters the saving is a rounding error (for a vocabulary of 200k and $d = 8{,}704$, $V \times d \approx 1.7$B, under 0.1% of the total; derived with an assumed V), so Base-2 lets the two tables differ, which gives the output layer freedom the input embedding does not need.

Terms

query, key, value heads The parallel attention channels of Part One. In GQA the numbers of query heads and of key/value heads differ.

KV cache Stored keys and values of past tokens, kept during generation so each new token attends to them without recomputation. Its size per token is $2\times(\text{layers})\times h_{kv} \times d_h \times$ bytes.

multi-query attention (MQA) One key/value head shared by all query heads (Shazeer 2019); the extreme of GQA.

sliding window The fixed number w of most recent tokens a local layer can attend to. Not stated in the report for Base-2.

unembedding The output matrix that maps the final residual vector to a score per vocabulary entry.

weight tying Using one matrix for both embedding and unembedding.

Intuition

The attention design is a budget allocation. Local layers are cheap to cache, so they get many K/V heads; global layers are expensive to cache, so they get two. Five local layers to one global is the same instinct as the MoE: do the cheap thing almost everywhere and the expensive thing only where it is needed. The question this raises, which the report does not answer, is how much long-context quality two K/V heads in seventeen layers can carry.

4 The three initiatives (slide 9)

4.1 Slide 9: Three initiatives, three sections of this deck

The report credits MAI-Base-2 to scaling science, model–infrastructure co-design and data. The deck follows that order.

What the slide shows Three colored cards. Green, *Scaling science*: scaling ladders that account for repeated training data (source scaling), and empirical scaling laws for learning rate, weight decay and dropout; Figures 2, 3, 4, 5; footer stages Ladders and Hyperparameters. Orange, *Model–infrastructure co-design*: an efficiency-gain metric in FLOPs and in wall-clock time, multi-round dropless MoE, Muon resharded for 16k GPUs, activation gradient clipping; Figures 6, 7, 8, 9; footer stages MoE systems, Muon, Stability. Yellow,

Scaling data: expanded in-house pipelines, embedding-based diversity sampling, a 50T-token mixture and native vision data; Figure 10; footer stage Data.

Details The report says three initiatives were particularly important: scaling science (scaling ladders that account for repeated training data, alongside empirical hyperparameter scaling laws); model–infrastructure co-design (improved MoE and Muon implementations, alongside activation gradient clipping); and scaling data (expanded data pipelines and the integration of vision data). The report's sections follow the same order, and so does the rest of the deck: Sections 5–6 of these notes are scaling science, Sections 7–9 are co-design, Section 10 is data.

One way to see the three as one program: the ladder (science) decides what the model should be; co-design makes the chosen model affordable on the hardware that exists; data is the input the whole apparatus consumes. The report's three closing principles (slide 33), learned from first principles, simple methods scale best, and scientific rigor over shortcuts, are the values behind the three initiatives.

Terms

source scaling Restricting the unique data on small ladder rungs so that every rung repeats its data as many times as the full run does. Slide 12.

efficiency gain (EG) The report's metric for a recipe change: the compute saved at equal evaluation loss, measured in FLOPs or in wall-clock time. Slide 19.

model–infrastructure co-design Choosing the model and the training systems together, so that what is best on paper is also fast on the machines at hand.

diversity sampling Preferring novel documents, judged by embedding distance, when selecting training data. Slide 30.

Intuition

The slide is a table of contents, but the grouping is itself a claim: the report attributes a frontier model to a scientific method, an engineering discipline and a data operation, each with its own section and its own figures. Few frontier reports separate the three so cleanly, which is what makes this one teachable.

5 Scaling ladders (slides 10–14)

5.1 Slide 10: Scaling ladders: one number sets the whole model

A ladder is a family of models with one fixed recipe. Each rung is named by its layer count L, and L fixes the rest.

What the slide shows A horizontal log axis of training compute from 10^{21} to 10^{25} FLOPs, with five green rungs standing on it at increasing heights, L18 ($d = 1{,}536$), L24 (2,048), L30 (2,560), L36 (3,072) and L42 (3,584), their positions read off Figure 2 and approximate; a dashed line extrapolating two decades to the right; and a tall orange rung, L102, $d = 8{,}704$, 96 heads, MAI-Base-2. A panel states the ladder's rules and checks that $L = 102$ reproduces the big model. The right-hand text: every design question is answered by re-running the rungs and checking that the gain holds as L grows; the hard part is making small rungs behave like the big model.

Details The report says it evaluates model changes with a scaling ladder, a family of models of different sizes with a fixed training recipe; each rung is named after its number of layers L, and L alone determines the rest of the architecture. Its two examples: the hidden size is $(256/3)L$, and the number of query heads is L

rounded to the nearest multiple of 16. Models of increasing size are trained holding the tokens per active parameter (TPP) constant.

The check on the slide: $L = 102$ gives $d = (256/3) \times 102 = 8{,}704$ exactly, and 102 rounded to the nearest multiple of 16 is 96 (102 is 6 from 96 and 10 from 112), which are MAI-Base-2's hidden width and query-head count. So the big model is itself a rung of the ladder, the top one. The rungs' widths on the slide follow the same rule: 18 → 1,536, 24 → 2,048, 30 → 2,560, 36 → 3,072, 42 → 3,584 (derived). The head rule gives 16 heads at L18, 32 at L30 and L36, 48 at L42; at L24 the nearest multiple of 16 is ambiguous (16 and 32 are equidistant) and the report does not say which way it rounds. Note that d/heads is not an integer at $L = 102$ ($8{,}704/96 \approx 90.7$), so the head dimension must be set by a separate rule the report does not state.

Why one number. A ladder is useful only if its rungs differ in nothing but scale. If each rung were designed by hand, a change that helped at L24 might fail at L42 because of some other difference between the two. Tying every architectural quantity to L removes that freedom: the aspect ratio ($d/L = 256/3 \approx 85$), the head count and, through fixed TPP, the training horizon all move together. Bhagia et al. (2024) describe the same discipline for OLMo's ladders; DeepSeek-V3's report shows 16B and 230B pilots before its 671B model; the method is standard, and the report's contribution is in how it handles repeated data (slide 12).

The compute span. The five rungs span roughly 2×10^{21} to 2×10^{23} FLOPs (read off Figure 2), two decades; MAI-Base-2 sits at 2.4×10^{25}, two decades further. The ladder's top rung therefore cost about one hundredth of the final run, and its bottom rung about one ten-thousandth. Rungs are cheap enough to run many times, which is what makes them an experimental instrument.

Terms

rung One model in a scaling ladder, named by its layer count: L18, L24, . . . , L102.

aspect ratio Hidden width over depth, d/L; fixed at $256/3$ by the ladder's rule.

tokens per parameter (TPP) Training tokens divided by active parameters; the ladder holds it constant across rungs. Slide 11.

fixed recipe Everything except size held constant: optimizer, schedule, data mixture, numerics, initialization.

extrapolation Predicting the top rung's loss from a curve fitted through the lower rungs; Figure 2.

Intuition

A wind tunnel. You cannot fly the airliner to test a wing change, so you build scale models, and the models are only informative if they are faithful in the quantities that matter (for aircraft, Reynolds number; for language models, tokens per parameter and data repetition). The ladder's rules are the scaling laws of the wind tunnel, and the two slides after this one are about which similarity conditions must hold.

5.2 Slide 11: Background: scaling laws and tokens per parameter

Loss falls as a power law in compute. How to split compute between parameters and tokens is a separate choice.

What the slide shows An illustrative curve of loss against tokens per parameter at fixed compute, on a log axis from 2 to 3,000: lowest near TPP ≈ 20, the compute-optimal point, rising slowly to the right. An orange dot marks MAI-Base-2 at 50T/80B ≈ 625. The background panel: power laws (Kaplan et al. 2020), Chinchilla (Hoffmann et al. 2022) and its roughly 20 tokens per parameter, and why frontier models train far past that point. The curve is illustrative; the slide says so.

Details Kaplan et al. (2020) found that the loss of a language model falls as a power law in each of

parameters N, data D and compute C when the other two are not limiting, over seven orders of magnitude of compute. Hoffmann et al. (2022, Chinchilla) asked how to split a fixed compute budget between N and D and found, with three methods, that they should grow together, roughly $N \propto C^{1/2}$ and $D \propto C^{1/2}$, which at their scales meant about 20 tokens per parameter. Their parametric fit,

$$\mathscr{L}(N,D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}},$$

with E the irreducible loss, is the form most later work uses; the two power-law terms are the penalties for too few parameters and too few tokens. At fixed $C = 6ND$ the loss as a function of D/N has a minimum, which is the compute-optimal TPP, and rises on either side; the slide's curve is a sketch of this.

MAI-Base-2 trains at TPP $= 50 \times 10^{12} / 80 \times 10^{9} = 625$ (derived), thirty times the Chinchilla-optimal ratio. This is deliberate and now universal at the frontier. Chinchilla's optimum minimizes training compute for a target loss, but a model is trained once and served many times, and serving cost scales with active parameters, not with the tokens it was trained on. A smaller model trained longer costs more to train for the same loss but less to serve; Sardana and Frankle (2023) formalize the trade-off. Over-trained models also extrapolate well: Gadre et al. (2024) show that laws fitted at high TPP remain reliable.

Note that Figure 4 (slide 17) shows a series at TPP 677 alongside 30, 100 and 300, presumably the final run's ratio computed with a parameter count a little below 80B (50T/677 $\approx$ 74B; derived); the report does not say what count it uses, and 625 and 677 should be read as the same number under two conventions.

Terms

scaling law An empirical power-law relation between loss and a resource (parameters, data, compute), used to predict large runs from small ones.

power law $y = aX^{-\alpha}$: a straight line on log-log axes. Here $\mathscr{L} - E \propto N^{-\alpha}$ etc.

compute-optimal The allocation of a fixed compute budget between parameters and tokens that minimizes loss; Chinchilla's ≈ 20 tokens per parameter.

irreducible loss (E) The entropy of the text itself, which no model can go below; the floor in the Chinchilla fit.

over-training Training far beyond the compute-optimal TPP to get a smaller model for the same loss; cheaper to serve.

Intuition

Compute-optimal is the cheapest way to reach a loss once; it is not the cheapest way to reach it and then answer a billion queries. The frontier trains small-for-its-budget models for a very long time because inference, not training, is where most of the money goes. The ladder has to respect this: it is built at the same high TPP as the final run, not at Chinchilla's.

5.3 Slide 12: Source scaling: repeat the data alike on every rung

Small runs usually see each example once, the big run many times. This ladder repeats data alike on every rung.

What the slide shows Three rows of blocks. The full run: 50T tokens drawn as 8 blocks, the first dark (unique data) and seven light (repeats), captioned "8 passes over 6.25T unique tokens". A 5T rung with source scaling: the same 8-block pattern at one tenth the width, "one tenth the tokens, the same 8-fold repetition". A 5T rung without source scaling: 8 dark blocks, "no repetition: a different, easier problem". The reading:

fixed TPP and fixed repetition fix the unique data per parameter on every rung; without that, the report says, small experiments favor under-regularized choices that overfit at scale. The background panel: Muennighoff et al. 2023 on data-constrained scaling, why small runs mislead, and a pointer to Figure 10.

Details The report says: to account for multi-epoch training at smaller scales, it uses a technique it calls source scaling. On the lower rungs, source scaling artificially restricts the set of unique samples the model trains on, so that each dataset is repeated the same number of times regardless of the training horizon; for example, if a dataset is repeated 8 times over the full 50T-token run, a 5T-token run will loop 8 times over just 1/10 of that data. Under a fixed TPP this keeps the ratio of unique data to parameters fixed, which helps results transfer across scales; without it, experiments at smaller scale can favor under-regularized architecture and hyperparameter choices that overfit at larger scales. The report says including source scaling was key to scaling up to 2.8T parameters reliably.

The slide's "8 passes over 6.25T" is an illustration of the report's example applied to the whole corpus, not a statement of the corpus size: the report describes repetition per dataset ("each dataset"), and Figure 10 shows that different sources are repeated by different amounts. The mechanism is what matters. A frontier run of 50T tokens cannot use 50T unique tokens of good data; it repeats its better sources several times. A small rung that sees 5T unique tokens once is in a different regime: it never overfits, so it never rewards regularization, and a change that helps only under repetition (more dropout, more weight decay, a narrower model) looks useless on it. Worse, a change that *hurts* under repetition (a bigger expert, a lower weight decay) can look good. Source scaling puts every rung in the repeated regime of the final run.

Background. Muennighoff et al. (2023) trained models on repeated data and fitted a scaling law in which repeated tokens have decaying value: up to about four passes, repeated tokens are nearly as good as new ones; beyond that, their value decays toward zero, and extra compute is better spent on parameters than on more passes. Hernandez et al. (2022) found that even a small fraction of highly repeated data can hurt disproportionately. So repetition is a regime with its own law, and an experiment in the one-pass regime does not predict it. The dropout law of slide 17, with dropout growing with model size, is a direct consequence: on a source-scaled ladder the bigger rungs overfit more and the fit says so.

Terms

epoch, pass One complete traversal of a dataset. Eight passes means each example is seen eight times.

unique tokens The size of the corpus counted without repeats; the report's measure of corpus growth (36% from Base-1 to Base-2).

data-constrained scaling Scaling laws for the case where unique data, not compute, is the binding limit (Muennighoff et al. 2023).

regularization Anything that reduces overfitting: dropout, weight decay, smaller models, more data. Under repetition it matters more.

under-regularized A configuration that fits the training data too closely; invisible when every example is seen once.

Intuition

Studying for an exam from a textbook you will read eight times is a different task from reading eight different textbooks once. A method that works for the second (read fast, skim, do not take notes) may fail for the first. Source scaling makes every practice exam use the same number of re-readings as the real one, so the study methods that win in practice are the ones that win on the day.

5.4 Slide 13: Figure 2: the ladder forecast the final model

Five rungs across two decades of compute, fit once, extrapolated two more. The prediction landed on the actual value.

What the slide shows The report's Figure 2. The horizontal axis is training compute in FLOPs, logarithmic, from 10^{21} to beyond 10^{25}. The vertical axis is an evaluation score whose tick labels run from 1.270 at the top to 0.803 at the bottom; the report does not name the quantity (lower is better, and its scale suggests a weighted held-out loss). Five green dots are the ladder's final checkpoints, labelled L18 through L42; a solid curve is the ladder fit through them; a dashed curve extrapolates it to the right; a star at about $10^{25.4}$ FLOPs is the predicted final value for MAI-Base-2, and an open circle at the same place is the actual final value. The star and circle coincide, near 0.80. The slide's text: what is plotted, the test, and why it matters; a background panel on what a ladder fit assumes.

Details Reading the points off the figure (approximate): L18 at about 2×10^{21} FLOPs scores 1.27; L24 at 10^{22}, 1.13; L30 at 3×10^{22}, 1.05; L36 at 7×10^{22}, 1.00; L42 at 2×10^{23}, 0.95; the prediction and the actual value at about 2.5×10^{25}, 0.80. The report says the ladder "accurately forecast MAI-Base-2's performance" and the figure's legend lists the five series: ladder final checkpoints, ladder fit (L18–L42), extrapolation, predicted final, actual final.

What the fit assumes. The usual form for loss against compute is a power law with a floor, $\mathscr{L}(C) = E + AC^{-\alpha}$ (Kaplan 2020, Hoffmann 2022); on the figure's semi-log axes a pure power law without a floor would be a straight line, and the visible curvature is consistent with a floor. The report does not state its functional form or the fitting procedure. Whatever the form, an extrapolation over two decades assumes that the rungs and the final model lie on one curve, which is exactly what the ladder's rules (slide 10) and source scaling (slide 12) are designed to guarantee: same recipe, same aspect ratio, same TPP, same repetition.

Why it matters. The report says MAI-Base-2 ran on 16k GPUs for 38 days. The decision to commit that compute rested on the dashed curve, drawn from runs each of which cost at most about one hundredth of the final run (derived from the figure's positions), and the actual final value landed on the forecast. This is the empirical justification for everything else in the Ladders and Hyperparameters sections: changes tested on the rungs were trusted at scale because the rungs predicted the scale.

Two caveats worth teaching. The figure shows one scalar; a model can hit its forecast loss and still differ in specific capabilities. And a forecast that lands exactly is partly luck: Choshen et al. (2024) find that typical ladder extrapolations err by a few percent in loss, which here would be invisible at the figure's resolution but not negligible.

Terms

ladder fit The curve fitted through the rungs' final evaluations as a function of training compute.

predicted vs actual final The star (fit extrapolated to the final run's compute) and the open circle (the final run's measured score) on Figure 2.

power law with a floor $\mathscr{L} = E + AC^{-\alpha}$; the floor E is the irreducible loss.

final checkpoint The model at the end of its schedule, after learning-rate decay; the evaluation the ladder compares. Mid-run checkpoints at high learning rate are not comparable.

Intuition

The figure is the receipt for the ladder. Five small experiments, a curve, and a bet of 16,000 GPUs for 38 days that the curve would hold for a model a hundred times more expensive than the largest experiment. It held. Ask what would have happened had it missed by five percent: the report says nothing, and that silence is the cost of a frontier run.

5.5 Slide 14: Figure 3: how much compute is the recipe worth?

Project an external model onto MAI's curve. The horizontal gap is the compute the recipe saves at equal quality.

What the slide shows The report's Figure 3 on the left: four panels, Held-Out Code, QA, STEM and Math, each with training compute in FLOPs on a log horizontal axis (10^{21} to 10^{25}) and bits per byte on the vertical axis. In each panel the green ladder points L18–L42 and the point "L102 / MAI-Base-2" at about 2×10^{25} lie on a fitted curve. Two external points are placed at their own compute: MAI-Base-1 (an orange diamond at about 6×10^{24} FLOPs) and DS-V4-Pro (a gray square at about 10^{25}). From each, a horizontal arrow runs left to where the curve reaches the same BPB, labelled with the ratio: 12.7× and 12.3× on code, 6.3× and 4.2× on QA, 8.3× and 8.3× on STEM, 7.5× and 6.6× on math. In the middle, a sketch of the construction; at the right, the table of ratios and the method.

Details The report says direct comparisons between models conflate the quality of the training recipe with differences in size and compute. To isolate the recipe it estimates how much compute its own recipe would need to match an external model: for each of the four held-out evaluations, it fits an independent scaling law of BPB against training FLOPs through the ladder, including MAI-Base-2's final point so that the matching point falls within the fitted range rather than beyond it, finds the compute at which the curve reaches the external model's BPB, and divides the external model's training compute by it. The report says its recipe is 4.2–12.3× more FLOPs-efficient than DeepSeek-V4-Pro-Base, which it uses as the reference because it is the only released base checkpoint with more than one trillion parameters, and 6.3–12.7× more efficient than MAI-Base-1, its own model from June. It attributes the gain to the quality and relevance of its data and to the efficiency gains from its modeling choices.

The ratios by evaluation (from the figure): versus DeepSeek-V4-Pro-Base, 12.3× (code), 4.2× (QA), 8.3× (STEM), 6.6× (math); versus MAI-Base-1, 12.7×, 6.3×, 8.3×, 7.5×. The spread across evaluations is itself informative: the code gap is largest, and the held-out code set is MAI's own codebase, which the data section (slide 31) shows is where the mixture is heaviest (45% code).

What the method needs and what it does not show. It needs the comparator's training compute, which for DeepSeek-V4-Pro-Base the deck bounds from the model card at $\geq 9 \times 10^{24}$ FLOPs (slide 5b), consistent with the figure's placement near 10^{25}. It does not say that MAI-Base-2 was trained with 4–12× *less* compute than DeepSeek: it was trained with about 2.5× *more* (derived, slide 5b). The claim is about quality per FLOP along MAI's curve, not about the two final runs' budgets; the report's own phrasing, "we have put that saving into training a larger model", makes the same point.

Two assumptions the figure rests on. The projection reads the comparator against MAI's curve, so it assumes that MAI's ladder is the right curve to project onto; a comparator with a different data mix would have a different curve, and its "equivalent compute" on MAI's curve measures the combination of its recipe and its data against MAI's. And the ratios depend on the comparators' FLOPs counts, which for DeepSeek come from a model card and for the post-trained comparators are not available at all, which is why only two models appear in Figure 3.

Terms

FLOPs-efficiency The ratio of an external model's training compute to the compute MAI's recipe would need to reach the same BPB. A recipe-quality measure.

projection onto a scaling law Finding where a fitted loss-versus-compute curve reaches a given loss; the horizontal arrows in Figure 3.

reference model The external model a comparison is anchored to; here DeepSeek-V4-Pro-Base, chosen as the only base checkpoint above 1T parameters.

equal-quality compute The compute at which two recipes reach the same loss; the quantity the ratios compare.

Intuition

A horizontal ruler on a loss curve. If my curve reaches your loss at a tenth of your compute, my recipe is ten times as efficient, whatever either of us actually spent. Two things to keep straight: efficiency is measured along *my* curve, and it says nothing about who spent more. MAI-Base-2 spent more than DeepSeek and claims to have needed far less; both are true at once.

6 Hyperparameter laws (slides 15–18)

6.1 Slide 15: LR and weight decay: one knob and a half

Good configurations lie on a diagonal of the (LR, WD) plane; in rotated coordinates the diagonal becomes an axis.

What the slide shows Two grids of 25 dots. Left, axes log LR and log WD: the good configurations (purple dots) lie in a diagonal band where LR $\times$ WD is constant, and a rectangular grid puts only 7 of 25 runs in it. Right, axes log LR and log(LR $\times$ WD): the band is now a horizontal row, and every learning rate on the grid is probed within it. The caption says the band's position is illustrative. The background panel explains why the product matters: AdamW's decoupled decay turns the weights into a moving average with timescale $1/(\eta\lambda)$ steps; LR sets the step size and LR$\times$WD how far back the weights remember; and a scope note that Base-2 trains most matrices with Muon but the report states its sweep in LR and WD, as does the deck.

Details The report says it swept learning rate η and weight decay λ jointly because their effects are coupled: performance is most sensitive to the product $\eta\lambda$, so good configurations lie in a narrow diagonal band of the (η,λ) plane; a rectangular grid in (η,λ) would sample this band inefficiently, so it swept a grid in $(\eta,\ \eta\lambda)$ instead.

Why the product. AdamW (Loshchilov and Hutter 2019) applies weight decay as a multiplicative shrink separate from the gradient step,

$$\theta_{t+1} = (1-\eta\lambda)\,\theta_t - \eta\,u_t, \qquad u_t \text{ the Adam-normalized update,}$$

which is an exponential moving average of the updates: unrolling, $\theta_t \approx -\eta\sum_{s<t}(1-\eta\lambda)^{t-s}u_s$, a sum in which an update from $\tau = 1/(\eta\lambda)$ steps ago has been shrunk by a factor e^{-1}. So $\eta\lambda$ sets the memory horizon of the weights in steps, and η alone sets the size of each step (Wang and Aitchison 2024 develop this view and argue the horizon should be set as a fraction of training). Two quantities with two separate roles; the rotation of the grid simply aligns the axes with the roles.

The rotated grid. In the slide's sketch a 5×5 rectangular grid lands 7 runs in the band; the rotated grid, with $\log(\eta\lambda)$ as the second axis, lands a whole row of 5 in the band *and* spreads them across all five learning rates, so the sweep resolves the step size at the right horizon instead of wasting most runs on horizons that are far too long or too short. The exact band and counts are the deck's illustration; the report's actual grids are in Figure 4 (slide 16), where the vertical axis is indeed LR $\times$ WD.

Scope. MAI-Base-2 uses Muon for its non-embedding 2-D parameters (slide 24), and Muon has its own learning rate and weight decay (Liu et al. 2025 add decoupled weight decay to Muon for exactly this reason). The report presents the sweep in terms of LR and WD without saying which optimizer's; the laws of slide 17 are stated for the run as a whole. The deck follows the report.

Terms

learning rate (LR, η) The step size multiplying each update.

weight decay (WD, λ) A per-step multiplicative shrink of the weights toward zero, $(1-\eta\lambda)$ in AdamW; a regularizer and a memory horizon.

decoupled weight decay Applying the shrink directly to the weights rather than adding $\lambda\theta$ to the gradient before Adam's normalization; the W in AdamW.

EMA timescale $\tau = 1/(\eta\lambda)$ steps: how far back the weights remember their updates.

hyperparameter sweep Training many small runs over a grid of settings and picking, or fitting, the best.

Intuition

A car with a throttle and a brake that are nearly redundant is hard to drive; turn one into "speed" and the other into "how long before you forget where you were going" and each does one thing. The rotation is a change of variables that makes the two hyperparameters independent, which is why the report could fit laws for them separately on the next two slides.

6.2 Slide 16: Figure 4, top: fit the loss surface, do not pick the best run

Each panel is one model size and token budget. A 2-D quadratic is fit to the runs; its analytic minimum is the answer.

What the slide shows The top two rows of the report's Figure 4: six panels, L12 and L18 at TPP 30, 100 and 300. Each panel's horizontal axis is LR (log, 10^{-4} to 0.03) and its vertical axis is LR $\times$ WD (log, 10^{-6} to 0.1); the background color is the fitted quadratic surface of weighted evaluation NLL (the color bar runs from about 1.43 to 2.65), circles are the training runs sized by their weight in the fit, an open circle marks the empirical minimum (best single run), and a star marks the fit's analytic minimum. Each panel's header prints both minima. The slide adds a table of the fit minima and three readings: how to read a panel, why fit rather than pick, and the trend that more steps lower both LR* and WD*.

Details The report says that for several combinations of model size and TPP it trained models along the search grid, evaluated them on its pre-training evaluations, fit a 2-D quadratic function to the evaluations and extracted the analytic minimum (η^*, λ^*). Its figure caption adds that each circle's color is its measured NLL, its size is its weight in the fit, and the weight decreases with the relative gap between the run's NLL and the minimum in that slice, so runs far from the optimum, where a quadratic is a poor model of the surface, count less.

The fit minima printed on the panels (the report's numbers):

	L12, TPP 30	L12, 100	L12, 300	L18, 30	L18, 100	L18, 300
LR* (fit)	0.00374	0.00213	0.00197	0.0045	0.00266	0.00168
WD* (fit)	0.869	0.501	0.462	1.16	0.651	0.298
LR (best run)	0.0012	0.00124	0.00205	0.00441	0.00426	0.00117
WD (best run)	2.21	0.815	0.524	1	0.305	0.432

The slide rounds the fit minima to two figures. The empirical minima are included here to make the slide's point: the best single run can sit a factor of three away from the fitted optimum in LR (L12, TPP 30) and in WD (L18, TPP 100), because it is one noisy sample on a grid point while the fit uses every run in the panel. A quadratic in $(\log\eta, \log\eta\lambda)$ is a parabolic bowl, the simplest surface with a minimum; fitting it is the same idea as fitting a parabola through three points to locate a peak more precisely than the best point.

The trend. Reading across each row, both LR^* and WD^* fall as TPP rises from 30 to 300: longer training wants smaller steps and a longer memory. Reading down a column, the larger model (L18) wants a slightly larger LR at TPP 30 and a smaller one at TPP 300. These are the patterns the bottom row of Figure 4 (slide 17) turns into laws. Note the products: $\eta^*\lambda^*$ is 0.00325 at L12/TPP 30 and 0.00091 at L12/TPP 300 (derived), and the law of slide 17 says this product is $17.04/\text{steps}$, so the two panels correspond to about 5,200 and 18,700 steps (derived), a ratio of 3.6 for a tenfold change in tokens, which implies the batch size also grew with TPP; the report does not state its batch sizes.

Terms

quadratic surface fit A second-degree polynomial in two variables fitted to measured losses; its minimum is found analytically.

empirical minimum The best single training run on the grid.

fit minimum The minimum of the fitted surface; interpolates between runs.

fit weight How much a run counts in the least-squares fit; here decreasing with the run's distance in loss from the slice's best.

weighted evaluation NLL The report's objective for the sweep: a weighted combination of negative log-likelihoods on its evaluation sets.

Intuition

Do not trust the winner of a noisy tournament; trust the shape of the standings. Twenty-five runs each carry noise, but a bowl fitted to all of them averages it out and can put its minimum between grid points. The deck's phrase for the slide, "fit the surface, do not pick the best run", is good general advice for any hyperparameter search.

6.3 Slide 17: Figure 4, bottom: the laws behind the hyperparameters

Fitted optima from many (size, TPP) slices, modeled as functions of training steps and tokens per parameter.

What the slide shows The bottom row of Figure 4: two scatter plots of fitted optima against $\log$ steps (8 to 12), with points labelled by rung (L12 to L42) and shaped by TPP (30, 100, 300, 677). Left, $\log(\mathrm{LR}^* \times \mathrm{WD}^*)$ from -5 to -9, all points on one straight line of slope -1, printed as $\mathrm{LR}^* \times \mathrm{WD}^* = 17.04/\text{steps}$. Right, $\log \mathrm{LR}^*$ from -5 to -7.5 on four parallel lines, one per TPP, printed as $\log \mathrm{LR}^* = -1.505 - 0.572 \log \text{steps} + 0.174 \log \mathrm{TPP}$. Below the figure the slide prints the three final laws and a note on model selection; the background panel gives both exponents a story.

Details The report says it modeled η^* and λ^* as functions of model size, TPP or other features such as $\log$ steps, and found by model selection that $\eta^*\lambda^*$ depends primarily on the number of training steps while η^* additionally depends on TPP. The empirical exponent of $\eta^*\lambda^*$ with respect to steps was very close to -1, which the report says matches the view that the decoupled weight-decay timescale should be a fixed fraction of training; the exponent of η^* with respect to steps was close to -0.5, which matches the rule of thumb that doubling the batch size multiplies the learning rate by $\sqrt{2}$. The final laws as the deck prints them:

$$\mathrm{LR}^* \times \mathrm{WD}^* = 17.04/\text{steps}$$
$$\log \mathrm{LR}^* = -1.51 + 0.175 \log \mathrm{TPP} - 0.573 \log \text{steps}$$
$$\text{Dropout} = \text{sigmoid}\big(0.526\,(\log \text{ActiveParams} - 27.2)\big)$$

The figure's own annotation prints the LR law with coefficients -1.505, -0.572 and 0.174; the deck's -1.51, -0.573 and 0.175 come from the report's typeset equations, which differ from the figure's labels in the third decimal. The report says it estimated the laws in small-scale experiments, validated them at an intermediate scale where they consistently outperformed previous heuristics, and then used them to set MAI-Base-2's hyperparameters.

Which logarithm. The left panel has $\log$ steps from 8 to 12 and $\log(\eta^*\lambda^*)$ near -5.2 at the left end; $\ln 17.04 - 8 = -5.16$, so the axes and the law are in natural logarithms (derived check); $e^8 \approx 3{,}000$ and $e^{12} \approx 163{,}000$ steps are the range of the sweeps.

The exponent -1. $1/(\eta^*\lambda^*) = \text{steps}/17.04$: the EMA timescale of the weights (slide 15) is $1/17.04 \approx 5.9\%$ of the run, whatever its length. Wang and Aitchison (2024) argue the timescale should be a fixed number of epochs; for a run that sees its data a fixed number of times, as a source-scaled ladder does, a fixed fraction of the run is the same statement.

The exponent -0.5. At fixed tokens, halving the steps doubles the batch. Malladi et al. (2022) derive for Adam-type optimizers that doubling the batch should multiply the learning rate by $\sqrt{2}$ to keep the training dynamics the same (the "square-root scaling rule"), so $\eta^* \propto \text{steps}^{-1/2}$ at fixed TPP is that rule. The small positive TPP exponent says that at fixed steps, more tokens per parameter want a slightly larger learning rate.

Dropout. The law says dropout grows with active parameter count through a sigmoid centered at $\log \text{ActiveParams} = 27.2$. The report states neither the unit of ActiveParams nor the log base, so the deck computes no value. If the count is in raw parameters and the log is natural, $e^{27.2} \approx 6.5 \times 10^{11}$ is the center, and the formula gives about 0.25 for 80B active and 0.18 for 35B (derived, under those two assumptions); with any other unit the formula gives values near 0 or 1, which is weak evidence for the assumptions. Dropout increasing with size is what source scaling predicts: bigger rungs at fixed TPP and fixed repetition overfit more.

Terms

hyperparameter scaling law An empirical formula giving an optimal hyperparameter as a function of scale variables (steps, TPP, size), fitted on small runs and applied to the large one.

model selection Choosing which variables enter a fitted law, by comparing fits with and without each.

training steps Number of optimizer updates; tokens divided by batch size.

square-root scaling rule For Adam-type optimizers, multiply the learning rate by $\sqrt{k}$ when multiplying the batch by k (Malladi et al. 2022); linear scaling is the SGD analogue (Goyal et al. 2017).

dropout Randomly zeroing a fraction of activations during training (Srivastava et al. 2014); a regularizer whose rate the report sets by a law.

sigmoid $\sigma(z) = 1/(1+e^{-z})$, which maps the real line to $(0,1)$; here used so the dropout rate stays a probability.

Intuition

Two laws, two stories, both older than the report: remember the last six percent of the run, and take steps that shrink like one over the square root of their number. The report's contribution is not the rules but the demonstration that fitting them on a source-scaled ladder gives numbers that beat hand-tuned heuristics at the intermediate scale, and then trusting them at the full scale without a sweep.

6.4 Slide 18: Figure 5: a run with no spikes

Training loss over the full horizon. MAI-Base-1 spiked early; MAI-Base-2 did not spike at all.

What the slide shows The report's Figure 5: two panels, MAI-Base-1 and MAI-Base-2, with the fraction of training completed (0 to 1) on the horizontal axis and training loss (0.6 to 2.0) on the vertical axis. Base-1's curve shows several sharp upward spikes in roughly the first 15% of the run, then a smooth descent to about 0.80. Base-2's curve descends smoothly from the start to about 0.72, with no spikes, and steepens slightly in the last part of the run. The slide's readings: what changed, that the two curves are not comparable in level, and why spikes matter; a background panel on where loss spikes come from.

Details The report says MAI-Base-2's training was smooth over the full training horizon, and credits that to its focus on careful scaling ladders and incremental refinement of its training recipe. The Stability section of the report adds the two concrete mechanisms, Muon and activation gradient clipping, which it says together keep the gradient norm in a much tighter range (Figure 7, slide 28).

Not comparable in level. The two models train on different mixtures (Figure 10: Base-2 adds vision data and changes every source's share), different token counts and different evaluation-weighting, so the fact that Base-2's curve ends lower than Base-1's (about 0.72 versus 0.80, read off the plot) is not by itself evidence of a better model; a mixture with more repeated or easier data would also end lower. Figure 1, on held-out data common to both, is the evidence. The late steepening of Base-2's curve is consistent with a learning-rate decay phase; the report does not describe its schedule.

Why spikes matter. A loss spike is a sudden jump in training loss, usually followed by partial recovery. The tokens around it are wasted, and if the model does not recover, the run must be rolled back to an earlier checkpoint and resumed, often with the offending batches skipped, which at 16k GPUs costs hours per incident; Base-2's 89% goodput would have been lower with rollbacks. Spikes also leave a mark: a model that spiked can end at a worse loss than one that did not, so smoothness is an efficiency gain as well as an operational one.

Where spikes come from. Wortsman et al. (2023) reproduce two large-scale instabilities in small models by raising the learning rate: attention logits grow until the softmax saturates, and output logits diverge; the fixes are QK-normalization (normalizing queries and keys before their dot product, as ViT-22B did) and a z-loss on the output softmax, and they show that the maximum stable learning rate falls with model size. PaLM (2022) reported about twenty spikes in its 540B run and handled them by restarting from a checkpoint and skipping the 200–500 batches around the spike. Base-2's approach is to clip the offending gradients at their source (slide 27) rather than to repair afterwards.

Terms

training loss The next-token negative log-likelihood averaged over a training batch, in nats; the curve a run is watched by.

loss spike A sudden rise in training loss, typically from a divergence inside the network or a bad batch.

rollback Restarting training from an earlier checkpoint after an instability, usually skipping the batches that caused it.

checkpoint A saved copy of the weights and optimizer state; the unit of recovery.

QK-normalization Normalizing query and key vectors before their dot product so attention logits cannot grow without bound (Henry et al. 2020; ViT-22B).

z-loss An auxiliary loss penalizing the log of the softmax normalizer, which keeps output logits from drifting (PaLM; also used in MoE routers by ST-MoE).

learning-rate decay The final phase of a schedule in which the learning rate is lowered; the loss typically drops noticeably during it.

Intuition

Two heart monitors. One shows arrhythmia early and then settles; the other is regular from the first beat. The second patient is not necessarily healthier (different diet, different scale), but the absence of events is itself the finding: 38 days, 16,000 GPUs, and nothing to roll back. The slides on Muon and clipping are the explanation; this slide is the symptom.

7 MoE systems (slides 19–23)

7.1 Slide 19: Co-design: score every idea in FLOPs and in wall-clock time

Efficiency gain (EG): the compute a candidate needs to reach the baseline's loss, relative to the baseline. Two clocks.

What the slide shows A quadrant diagram with $\mathrm{EG}_{\mathrm{FLOPs}}$ (does it save compute?) on the horizontal axis and $\mathrm{EG}_{\mathrm{time}}$ (does it save hours today?) on the vertical axis, each divided at 1. Top right, *adopt*: wins on both clocks. Bottom right, *invest in infrastructure*: a better model with slower kernels, the example being MoE-every-layer in the Base-1 era. Top left, *cheaper, not better*: an approximation that runs fast. Bottom left, *drop*. The text: decisions follow $\mathrm{EG}_{\mathrm{time}}$, which reflects the infrastructure as it is; $\mathrm{EG}_{\mathrm{FLOPs}}$ is the map showing where to invest; three results of this loop are MoE in every layer, Muon at 16k GPUs and activation gradient clipping. The background panel defines MFU and goodput.

Details The report says training frontier models within a compute budget requires co-designing models and infrastructure for both evaluation quality and computational efficiency, and that it uses an efficiency gain (EG) metric to guide decisions: the ratio of compute required by a candidate to reach the same evaluation loss as the baseline. Compute can be measured in FLOPs ($\mathrm{EG}_{\mathrm{FLOPs}}$) or in wall-clock time ($\mathrm{EG}_{\mathrm{time}}$), and the two are complementary. The report relies mainly on $\mathrm{EG}_{\mathrm{time}}$, since it reflects the current state of the infrastructure; $\mathrm{EG}_{\mathrm{FLOPs}}$ measures a prototype's potential and guides future infrastructure development: a change that wins on FLOPs but loses on wall-clock time shows where to invest.

The metric, made explicit. Take a baseline recipe and a candidate that differs in one change; train both (on the ladder) and record compute against loss; find the compute at which each reaches a chosen loss. The report phrases EG as a ratio of the candidate's compute to the baseline's; Figure 8 reports clipping as a positive percentage gain, so the deck orients the metric so that larger is better:

$$\mathrm{EG} = \frac{C_{\mathrm{baseline}}(\mathscr{L})}{C_{\mathrm{candidate}}(\mathscr{L})} \text{ at equal loss } \mathscr{L}; \quad \text{gain} = \mathrm{EG} - 1; \quad C \text{ in FLOPs or in GPU-hours.}$$

This is the same construction as Figure 3's projection (slide 14), applied to two versions of one recipe rather than to two models. It is a ratio of compute, not a difference of losses, which makes gains comparable across scales and additive in log space.

Why two clocks. FLOPs are a property of the model; time is a property of the model on today's kernels, interconnect and parallelism. A change can lower the FLOPs needed for a loss and still make the run slower if its kernels are immature (the MoE-every-layer example of slide 20). The report's rule, decide by time but map by FLOPs, means that the model shipped is the one that is fastest now, while the FLOPs score tells the systems team which kernels would pay off most. Once they do, the candidate moves to the top-right quadrant and gets adopted, which is exactly the history of MoE-every-layer between Base-1 and Base-2.

MFU and goodput. MFU (PaLM 2022) is achieved model FLOP/s divided by the hardware's peak; the report normalizes by the GB200's 2.5 PFLOPS BF16 peak and excludes rematerialization and optimizer FLOPs, giving 21% for the run. Goodput is the fraction of wall-clock time in which training advanced, 89%.

EG_{time} depends on both: a change that lowers MFU (more small kernels) or goodput (more failures) can lose on time while winning on FLOPs.

Terms

efficiency gain (EG) Compute saved at equal loss by a recipe change, as a ratio to the baseline; EG_{FLOPs} counts FLOPs, EG_{time} counts wall-clock time.

baseline vs candidate The current recipe and the recipe with one change; EG compares them.

wall-clock time Elapsed real time, which includes communication, idle time and failures; what a GPU-hour budget actually buys.

kernel A GPU program for one operation (a matrix multiply, an all-to-all); kernel quality determines how much of the peak a model attains.

co-design loop Propose a change, measure both EGs, adopt if time wins, invest in kernels if only FLOPs wins.

Intuition

Two stopwatches on every experiment: one counts arithmetic, the other counts hours. The arithmetic watch tells you what the idea is worth in principle; the hour watch tells you what it costs you today. Ship by the second, plan by the first. The gap between them is a to-do list for the systems team, and the next four slides are three items from that list, done.

7.2 Slide 20: Scaling MoE: two inefficiencies, then MoE in every layer

MoE-every-layer always won on FLOPs. Fixing the MoE implementation made it win on time as well.

What the slide shows Two bars, MFU of the MoE layers before (24%) and after (35%), with the note that the relative gain is +46%: at the same model FLOPs, 1.46× the throughput in those layers. Two numbered inefficiencies: (1) dynamic routing fragmented memory, since buffers sized for rare extreme imbalances tied up memory that could have cut activation recomputation; (2) finer experts, more overhead, since 512 small experts mean many small GEMMs and more launches, synchronizations and collectives per layer. The consequence: MoE-every-layer moved from "invest" to "adopt". A background panel explains activation recomputation and points to the next three slides.

Details The report says its past experiments showed that MoE in every layer has a consistent EG_{FLOPs} advantage over alternating dense FFN and MoE layers, but inefficiencies in its MoE implementation made MoE every other layer the EG_{time}-optimal choice for MAI-Base-1. For MAI-Base-2 it identified two inefficiencies behind the suboptimal MoE MFU: dynamic routing increased memory fragmentation and tied up memory for rare, extreme expert load imbalances, memory that could otherwise reduce activation recomputation; and adding more, finer-grained experts increased per-expert overhead. Addressing these raised the MFU of the MoE layers from 24% to 35%, which allowed MoE in every layer for MAI-Base-2.

The arithmetic. $35/24 = 1.46$ (derived): with the same FLOPs per layer, the MoE layers run 1.46× faster. Since Base-1 had MoE in only half its layers and Base-2 in all 102, the systems gain had to be large enough to make the all-MoE model, which does more MoE work per token, faster per unit of quality than the alternating one. Note that even 35% for the MoE layers is modest, and the whole-run MFU of 21% is lower still: the run average also absorbs attention, communication that is not overlapped, pipeline bubbles, rematerialization and the optimizer step, none of which count as useful FLOPs under the report's definition.

Inefficiency 1, memory. Routing is data-dependent: the number of tokens each expert receives varies from batch to batch and layer to layer and is known only after the router runs. If the buffers that hold each expert's

tokens are allocated for the worst case, most of that memory sits idle most of the time; if they are allocated dynamically, the allocator fragments. Either way, memory that could have stored activations is unavailable, so more activations must be recomputed in the backward pass, which costs compute. The multi-round dropless design of slide 22 fixes the memory at a tunable capacity.

Inefficiency 2, overhead. A GEMM of a few thousand tokens by a 4,096-by-8,704 expert is small for a GB200; with 512 experts per layer the work is many such small kernels, each with a launch cost, and the dispatch and collect communication for each adds synchronization. Slide 23 lists the four changes that cut this.

Rematerialization. Storing every activation of a 102-layer, 8,704-wide model for the backward pass exceeds GPU memory at any useful batch, so training recomputes some activations from checkpoints during the backward pass (Chen et al. 2016; Korthikanti et al. 2022 on how to minimize it for transformers). The report's MFU definition excludes these FLOPs, so they are pure overhead; memory freed from MoE buffers buys them back.

Terms

MoE-layer MFU Utilization measured within the MoE layers alone; 24% before and 35% after the report's fixes.

memory fragmentation Free memory split into pieces too small to use, caused by allocating and freeing buffers of varying size.

load imbalance Unequal token counts across experts in one batch; the extremes are rare but set the worst-case buffer size.

GEMM General matrix multiply, the kernel that does an expert's work; efficiency falls when the matrices are small.

kernel launch overhead Fixed cost per GPU kernel, independent of its size; dominates when kernels are small.

collective A communication operation involving many GPUs (all-to-all, all-reduce, all-gather).

Intuition

The model wanted to be all-MoE two years earlier; the kernels said no. Slide 19's map said where to invest, the investment raised the MoE layers' utilization by half, and the model the FLOPs clock preferred became the one the wall clock preferred. This is co-design as the report means it: a modeling decision unblocked by systems work, with a number attached.

7.3 Slide 21: Background: the capacity problem in expert parallelism

Routing is data-dependent, so some experts get far more tokens than others. Every MoE system must decide what then.

What the slide shows A histogram of tokens routed to eight experts in one batch (illustrative), with a dashed capacity line; two bars, E3 and E6, overflow it, their excess shaded. To the right, four options: drop the overflow (capacity factor with token dropping, as in GShard and Switch: fixed memory, but dropped tokens skip the layer; MAI measured worse models); allocate for the worst case (never drops, but the memory idles); dropless with variable memory (MegaBlocks 2022: block-sparse kernels process whatever arrives, no drops, but memory moves with the routing); and MAI's choice, dropless with fixed memory (fixed capacity per expert, overflow handled in extra rounds). The footer: memory on GPUs is allocated ahead of time, but token counts are known only after the router runs.

Details Expert parallelism (GShard 2020) places different experts on different GPUs. For each MoE layer, every token's hidden state is sent to the GPUs holding its chosen experts (dispatch, an all-to-all), processed there, and sent back (collect, another all-to-all). The number of tokens arriving at a GPU is the sum over its experts of the tokens routed to them, which varies with the data. GPU kernels and communication buffers need their sizes in advance; routing decides the sizes at run time. That is the capacity problem.

The classical answer is a capacity factor. With T tokens in the batch, E experts and k experts per token, the average load is kT/E tokens per expert; each expert is given a buffer of

$$C = \mathrm{CF} \cdot \frac{kT}{E} \text{ tokens}, \qquad \mathrm{CF} \text{ the capacity factor (1.0–2.0 in GShard and Switch);}$$

tokens beyond an expert's capacity are dropped: they skip the layer and pass through the residual connection unchanged. Dropping keeps memory and communication fixed, and the load-balancing loss keeps drops rare, but every dropped token is a token the layer did not process. The report says its experiments found that dropping tokens leads to worse performance.

Two dropless designs. Allocating every expert's buffer for the largest imbalance ever seen never drops, but the report notes it uses memory inefficiently under normal loads (inefficiency 1 of slide 20). MegaBlocks (Gale et al. 2022) reformulates the expert computation as a block-sparse matrix multiply whose blocks are allocated to match the actual routing, so no token is dropped and no buffer is padded; the price is that memory use varies with the routing, which at scale means it can spike. MAI's design keeps MegaBlocks' guarantee (nothing dropped) with the capacity-factor design's property (fixed memory) by handling overflow in time rather than in space: the next slide.

Why the balance of the router matters here. All four options get easier the more even the routing. Switch's auxiliary loss, DeepSeek-V3's bias-based balancing (Wang et al. 2024) and the report's batched load-balancing loss (slide 23) exist to keep the histogram flat; the capacity design handles what is left.

Terms

expert parallelism (EP) Distributing a layer's experts across GPUs and routing tokens to them over the network.

dispatch, collect The two all-to-all exchanges per MoE layer: tokens to their experts, results back to their source.

all-to-all A collective in which every GPU sends a distinct chunk to every other GPU; the communication pattern of expert parallelism.

capacity factor (CF) Buffer size per expert as a multiple of the average load; tokens beyond it are dropped.

token dropping Skipping an MoE layer for tokens that exceed their expert's capacity; they continue through the residual stream unprocessed.

dropless MoE Any scheme that processes every routed token; MegaBlocks is the standard reference.

block-sparse GEMM A matrix multiply over a grid of blocks of which only some are present; MegaBlocks' kernel for variable-size expert batches.

Intuition

A bank with eight tellers and a crowd whose queues are decided by the customers. Fixed-size queues with a "come back tomorrow" sign (drop); a hall large enough for every customer to crowd one teller (worst case, mostly empty); a hall whose walls move (MegaBlocks); or fixed queues and a second serving

round for whoever is left (MAI). The last keeps the building the same size and serves everyone, at the cost of occasionally running late.

7.4 Slide 22: Figure 6: multi-round dropless MoE, one forward pass

Three expert groups; groups 0 and 1 overflow. Compute and communication streams run side by side, time left to right.

What the slide shows The report's Figure 6, a timeline of one MoE layer's forward pass with time running left to right and two horizontal streams: a compute stream above and a communication stream below. Four phases are labelled across the top: Pre-expert compute, Round 0 (all groups have tokens), Round 1 (EG0 and EG1 have tokens), Post. In the compute stream: Router, then Shared experts and Weight Quant, then boxes EG0, EG1, EG2 for round 0, then EG0, EG1 and an empty EG2 for round 1, then LBL stats, a wait, and Combine. In the communication stream: a sequence of D (dispatch) and C (collect) boxes, staggered so that each group's dispatch precedes its compute and its collect follows it, overlapping the neighbors' compute. A legend marks overflow tokens, zero tokens, stalls and exposed communication. The slide's vocabulary notes and phase-by-phase reading are at the right; a panel below covers fixed memory and the backward pass.

Details The report says it groups the experts on each expert-parallel rank into one or more groups of a configurable size. Each expert group runs one or more dispatch → compute → collect rounds with a fixed per-expert token capacity; if an expert receives more tokens than it can hold, the group processes the overflow in additional rounds; to reduce the number of rounds, overloaded experts may use unused capacity from other experts in the same group. In the backward pass, fine-grained recomputation per expert group follows a similar multi-round schedule. This keeps memory usage static regardless of imbalance. Capacity is configurable: more capacity means more memory, less means potentially more rounds and extra compute; in practice it is configured so that steady state needs a single round.

Reading the figure. Pre-expert: the router runs and decides the routing; the shared experts, which every token uses and which need no dispatch, and the quantization of the expert weights to MXFP8 (slide 26) run on the compute stream while the first dispatch, which cannot be hidden, runs on the communication stream. Round 0: EG0 computes while EG1's dispatch arrives; then EG1 computes while EG0's collect departs and EG2's dispatch arrives; and so on, so that each dispatch and collect overlaps a neighbor's compute. Round 1: only EG0 and EG1 overflowed, but the round count is global, so EG2 takes its turn with zero tokens and its compute is skipped. Post: load-balancing-loss statistics, a wait for the last collect, which is the only other exposed communication, and the final combine that merges each token's expert outputs. The report's caption notes that MFU drops during extra rounds and that this is acceptable because steady state has only one round.

Overlap. The report says dispatch, compute and collect are pipelined across all rounds of all groups, so that all but the first dispatch and the last collect overlap with expert compute; in the forward pass the weight quantization and shared-expert compute cover the exposed communication; in the backward pass all expert communications are fully overlapped, the input dispatch with recompute of the same layer and the gradient collect with weight-gradient computation.

What "fixed memory" buys. The buffers for a group are sized for its capacity, once, and reused every round; no allocation depends on the data, so there is no fragmentation and no out-of-memory failure from an unlucky batch, and the memory not spent on worst-case buffers holds activations instead, cutting recomputation (slide 20). The cost, extra rounds, is paid only on the rare batches that overflow, and the group-level borrowing makes those rarer still.

Terms

expert group (EG) The experts on one expert-parallel rank, or a configurable subset of them; the unit of scheduling in the multi-round scheme.

round One dispatch → compute → collect cycle; steady state uses one, overflow adds more.

token capacity The fixed number of tokens an expert's buffer holds per round; tunable.

compute stream, communication stream Two independent GPU command queues, so that kernels and transfers run concurrently.

exposed communication Transfer time not hidden behind compute; here the first dispatch and the last collect.

weight quantization (Weight Quant) Converting expert weights to the FP8 training format before the GEMMs; overlapped with the exposed dispatch.

LBL stats Statistics for the load-balancing loss, gathered per layer and reduced once per step (slide 23).

Intuition

A relay in which every runner's handoff is hidden behind someone else's lap. Only the first and last handoffs show. When a runner is given more laps than planned, everyone runs one more round (even those with nothing to carry, who just step aside), and the track never has to be rebuilt mid-race. The design accepts a slow round now and then to never run out of track.

7.5 Slide 23: Cutting per-expert overhead: four changes

With 512 experts the work per expert is small, so launches, host round-trips and copies dominate unless removed.

What the slide shows At the left, four GPUs each holding a stack of experts, with arrows between them labelled "all-to-all over NVLink: dispatch tokens out, collect results back", and a note on expert parallelism: per expert, a small GEMM wrapped in communication. At the right, four cards: (1) group work per expert group: one batched GEMM and one collective per group, not per expert; (2) metadata stays on the GPU: token counts and offsets live on the device and the all-to-all is issued from the GPU; (3) symmetric memory over NVLink: tokens move directly between GPU memories without staging copies; (4) one load-balancing reduction: the auxiliary loss is batched across layers. The footer quotes the report: sparser models "without significantly degrading MFU".

Details The report says it reduces per-expert overhead in four ways. First, it improves GEMM and NVLink efficiency by grouping both computation and communication for each expert group, which reduces kernel-launch overhead and tail effects. Second, it reduces host–device synchronization by using device-resident token metadata and initiating all-to-all communication directly from the GPU. Third, it optimizes the all-to-all with symmetric memory to exchange tokens directly over NVLink without staging copies, improving throughput and reducing HBM contention when GEMMs run concurrently. Lastly, it batches the load-balancing loss across layers, replacing per-layer all-reduces with a single batched reduction. These optimizations, the report says, let it train sparser models without significantly degrading MFU.

Why each matters. (1) A grouped GEMM runs the matrix multiplies of all experts in a group as one kernel over a list of (tokens, expert) pairs, so there is one launch instead of dozens and the GPU's many streaming multiprocessors are kept busy together; grouping the communication likewise turns many small all-to-alls into one larger one, which is more efficient on the interconnect and removes the "tail" in which the last small transfer holds up the layer. (2) If the CPU has to learn how many tokens went where before it can

issue the transfer, every MoE layer pays a round-trip to the host, hundreds of microseconds in which the GPU idles; keeping the counts and offsets on the device and launching the collective from the device removes that stall. (3) An all-to-all normally copies tokens into a send buffer, transfers, and copies out of a receive buffer; with symmetric memory, the same virtual addresses are mapped on every GPU, so one GPU can write directly into another's buffer over NVLink, saving two copies per token and the memory bandwidth they consume, which matters because the GEMMs running concurrently need that bandwidth. (4) Load balancing needs the global token-per-expert counts, which requires an all-reduce; doing one per MoE layer means 102 small synchronizations per step, doing one for all layers means one.

The pattern across the four is the same: with fine-grained experts the useful work per expert is small, so anything with a fixed per-expert or per-layer cost, a launch, a host sync, a copy, a collective, becomes the bottleneck, and the fix is always to amortize it over a group or over the whole step. Tutel (Hwang et al. 2022) and DeepSpeed-MoE (Rajbhandari et al. 2022) describe the same bottlenecks and earlier versions of the same fixes.

Terms

NVLink NVIDIA's GPU-to-GPU interconnect; within a GB200 NVL72 rack, 72 GPUs share one NVLink domain, which is the fabric the all-to-alls use.

grouped GEMM One kernel that performs many independent matrix multiplies of differing sizes; the standard kernel for fine-grained MoE.

tail effect Idle time while the slowest of many small operations finishes.

host–device synchronization The CPU waiting on the GPU (or vice versa) to exchange a value; each costs the GPU a stall.

symmetric memory Buffers allocated at the same address on every GPU so that remote GPUs can read and write them directly.

HBM High-bandwidth memory, the GPU's main memory; its bandwidth is shared between kernels and transfers.

all-reduce A collective that sums a tensor across GPUs and gives every GPU the result; used for load-balancing statistics and for gradients.

load-balancing loss The auxiliary loss that keeps tokens spread across experts; its statistics need a cross-GPU reduction.

Intuition

When each job is tiny, the paperwork is the job. Four changes, each removing a fixed cost that 512 experts multiplied: one launch instead of many, no phone call to the CPU, no copying into and out of envelopes, one tally instead of one hundred and two. None is new in kind; the report's claim is that together they let the model go sparser without the MFU falling, which is what made MoE-every-layer affordable.

8 Muon (slides 24–25)

8.1 Slide 24: Background: Muon orthogonalizes the update

AdamW scales each weight separately. Muon treats the update as a matrix and equalizes its singular values.

What the slide shows A pipeline, left to right: the momentum matrix M of one weight matrix, drawn as a

grid; its singular values as a bar chart in which a few directions dominate; an arrow labelled "Newton–Schulz, 5 iterations, bf16"; the singular values after, all equal to 1, with $O \approx UV^\top$; and the update $W \leftarrow W - \eta\, O$ (scaled). Below, the iteration

$$X_0 = M/\|M\|_F, \qquad X_{k+1} = aX_k + b\,(X_kX_k^\top)X_k + c\,(X_kX_k^\top)^2X_k,$$

the effect (rare gradient directions move as much as dominant ones: steepest descent under a spectral-norm bound, Bernstein and Newhouse 2024), and MAI-Base-2's use (Muon for non-embedding 2-D weights, AdamW for the rest; more stable, large gains). The background panel: origin (Keller Jordan 2024; five bfloat16 Newton–Schulz steps suffice) and scale (Kimi K2 trained a 1T MoE with MuonClip; Liu et al. 2025 show the gain holds up at scale).

Details Adam and AdamW keep, for every scalar parameter, a running mean of its gradient and a running mean of its squared gradient, and step each parameter by the ratio: every weight gets its own step size, and the matrix structure of a weight matrix plays no role. Muon (Jordan 2024) keeps only the momentum M (a running mean of the gradient matrix) and, before applying it, replaces M by the nearest orthogonal matrix. If $M = U\Sigma V^\top$ is the singular value decomposition, the update is

$$O = UV^\top \quad \text{(all singular values set to 1)}, \qquad W \leftarrow W - \eta\, s\, O,$$

with s a scale factor. Computing the SVD of every weight matrix at every step is too expensive, so Muon approximates $UV^\top$ with the Newton–Schulz iteration on the slide: a fixed polynomial in X and $XX^\top$ applied five times, with coefficients chosen by Jordan so that the singular values are driven toward 1 quickly from any starting value in $(0, 1]$ rather than converging precisely (his coefficients are $a = 3.4445$, $b = -4.7750$, $c = 2.0315$). The iteration is three matrix multiplies per step on the matrix itself, runs in bfloat16, and needs the whole matrix, which is the systems problem of the next slide.

Why orthogonalize. A gradient matrix of a large layer typically has a few large singular values and a long tail of small ones: a few directions in weight space dominate the update, and the rare directions, which may carry information about rare tokens or features, barely move. Setting all singular values to 1 gives every direction the same step. Bernstein and Newhouse (2024) show that this is exactly steepest descent when the size of a step is measured by the spectral norm of the weight change (the largest factor by which the layer can stretch any input), which is the natural norm for a linear layer; Adam, by contrast, is steepest descent under a per-coordinate (infinity) norm that ignores the matrix structure. In this view Muon is not a heuristic but the "right" optimizer for matrix-shaped parameters, and the elementwise optimizers are the approximation.

Scope and scale. Muon applies to 2-D weights, the matrices inside attention and the experts; embeddings, the unembedding, norms and biases keep AdamW (Jordan's original recipe, and the report's). At scale, Liu et al. (2025) found two adjustments necessary: weight decay added to Muon, and the update rescaled so that its root-mean-square size matches AdamW's, so that one learning rate serves both; with these, Muon reached a given loss in about half the compute of AdamW on a 16B-total MoE. Kimi K2 (2025) trained a 1T-parameter MoE on 15.5T tokens with MuonClip, Muon plus a rescaling of attention weights (QK-clip) that caps attention logits, and reported no loss spikes. The report says Muon gave MAI-Base-2 more stable training dynamics and large efficiency gains over AdamW, and does not give the numbers.

Terms

Muon MomentUm Orthogonalized by Newton–Schulz (Jordan 2024): an optimizer that orthogonalizes each weight matrix's momentum before stepping.

momentum A running average of past gradients; the quantity Muon orthogonalizes.

singular value decomposition (SVD) $M = U\Sigma V^\top$ with U, V orthogonal and Σ diagonal; the singular values measure how much M stretches each direction.

orthogonalization Replacing M by $UV^\top$, the orthogonal matrix nearest to it; equalizes all singular values.

Newton–Schulz iteration A matrix polynomial iteration that converges to $UV^\top$; Muon uses five steps with coefficients tuned for speed rather than precision.

spectral norm The largest singular value of a matrix; the largest factor by which it stretches a vector.

steepest descent under a norm The update that decreases the loss most per unit of step size, where the step size is measured by a chosen norm; different norms give different optimizers.

MuonClip Kimi K2's variant: Muon plus QK-clip, which rescales query and key weights when attention logits exceed a threshold.

Intuition

Adam gives every weight its own gas pedal; Muon looks at the whole matrix and says: move every direction by the same amount. A gradient that shouts in two directions and whispers in a thousand becomes an update that speaks evenly in all of them. The whispering directions are often where the rare knowledge is, which is one story for why the gain in Figure 8-style experiments tends to grow with scale.

8.2 Slide 25: Muon on 16k GPUs: reshard, orthogonalize, reshard back

ZeRO shards momentum across data-parallel GPUs; Newton–Schulz needs whole matrices. Two all-to-alls bridge them.

What the slide shows Four step cards: (1) local momentum update: each GPU updates its FP32 shard of every momentum matrix and casts it to BF16; (2) all-to-all, batched: shards of same-shaped matrices are exchanged so each GPU owns a subset of whole matrices; (3) Newton–Schulz locally: a symmetry-aware kernel orthogonalizes each whole matrix on the GPU that holds it; (4) all-to-all back: updates return to their original shards, are cast to FP32 and applied. Two small pictures show one matrix in four shards and then whole on one GPU. Below: three more cuts (prefetching, cached resharding metadata, replicated small weights), and the accounting panel: Muon takes 2.1% of an iteration, 165 ms, so the iteration is about 7.9 s (derived); the team's communication-bound floor is 55 ms; the gap is stragglers, kernel dispatch, load imbalance and resharding copies.

Details The report says Muon orthogonalizes the momentum of each weight matrix with Newton–Schulz, an iterative algorithm of several matrix multiplications that requires the full matrix, which conflicts with ZeRO's sharding of momentum across the data-parallel dimension. Following prior work, it reshards the momentum during the optimizer step: each GPU updates its local FP32 momentum shard and casts it to BF16; an all-to-all exchange, batched over parameters of the same shape, gives each GPU ownership of a subset of full momentum matrices; each GPU orthogonalizes its matrices locally with a symmetry-aware Newton–Schulz kernel; a second all-to-all returns the updates to their original shards, where they are cast back to FP32 and applied to the weights.

Why the conflict exists. ZeRO (Rajbhandari et al. 2020) saves memory in data-parallel training by giving each of the N data-parallel GPUs only a $1/N$ slice of the optimizer state (and optionally of the gradients and weights) rather than a full copy; each GPU updates its slice and the full weights are reassembled by communication. For Adam this works because the update of each scalar depends only on that scalar's state. Muon's update of one entry depends on the whole matrix through $UV^\top$, so a GPU holding one slice cannot

compute its share. The resharding turns "every GPU holds a slice of every matrix" into "every GPU holds all of some matrices" for the duration of the orthogonalization, which is the same total data but the right partition.

The three further optimizations the report lists: overlapping communication and computation by prefetching multiple batches, so the all-to-all for the next group of matrices runs while the current group is being orthogonalized; caching and reusing the resharding metadata to avoid repeated indexing logic, which it says is especially useful in sparse MoE models with thousands of identically shaped parameters (512 experts per layer times 102 layers is 52,224 expert matrices of each shape; derived); and replicating selected small weights, such as the MoE router weights, across GPUs rather than sharding them, so the first communication phase is skipped and an all-gather after Newton–Schulz distributes the replicated updates.

The accounting. The report says that after these optimizations Muon accounted for 2.1% (165 ms) of MAI-Base-2's average training iteration time, and that it compared measurements with a theoretical peak it estimated at 55 ms, bottlenecked by communication for the final training setup, attributing the gap to stragglers, kernel dispatch overheads, load imbalance, and memory copies and allocations during resharding. From the two numbers, $165\,\text{ms}/0.021 \approx 7.9$ s per iteration (derived). The "symmetry-aware" kernel exploits that $XX^\top$ is symmetric, so about half its entries need not be computed; the report does not describe it further.

What this slide is an example of. The optimizer that won on the ladder (slide 24) would have been unaffordable at 16k GPUs with a naive implementation; the report's EG_{time} discipline (slide 19) required getting its cost down to a few percent before adopting it. The 55 ms floor is the honest end of the exercise: the team knows the implementation is three times slower than the interconnect allows and says why.

Terms

ZeRO Zero Redundancy Optimizer (Rajbhandari et al. 2020): partitioning optimizer state, gradients and weights across data-parallel GPUs instead of replicating them.

data parallelism Running the same model on many GPUs with different data, then averaging gradients.

shard One GPU's slice of a tensor under ZeRO partitioning.

resharding Changing which GPU holds which part of a tensor, by an all-to-all.

all-gather A collective in which every GPU ends up with the concatenation of all GPUs' pieces; used here for the replicated router updates.

prefetching Starting the communication for the next batch of work before the current one finishes, to overlap the two.

straggler A GPU that finishes its share late and holds up a collective; a major source of the gap between measured and ideal time.

iteration (step) time Wall-clock time of one optimizer step, forward, backward and update; about 7.9 s for MAI-Base-2 (derived).

Intuition

The whole-matrix operation and the slice-it-everywhere memory scheme want different partitions of the same data; the fix is to change partition twice per step, in bulk, and hide the cost behind other work. The number to remember is 2.1%: an optimizer that needs the full matrix was made nearly free, and the report also tells you how far from free it still is.

9 Numerics and stability (slides 26–29)

9.1 Slide 26: Numerics: from tensor-wise FP8 to MXFP8

Both models train matrix multiplies in 8-bit floats. Base-2 scales every block of 32 values separately, with no delay.

What the slide shows Left, Base-1's scheme: a 4×16 grid of values with one scale box for the whole tensor, computed from recent history; E4M3 for weights and activations, E5M2 for gradients; the scale lags the data by a step, so a sudden outlier can overflow. Right, Base-2's scheme: the same values in four blocks, each outlined with its own E8M0 scale computed on the spot; E4M3 everywhere. Two panels: the formats (FP8 per Micikevicius et al. 2022: E4M3 reaches 448 with 3 mantissa bits, E5M2 reaches 57,344 with 2; microscaling per Rouhani et al. 2023 and the OCP MX specification: blocks of 32 share an E8M0 scale, native on Blackwell; Kimi K3 ships MXFP4 weights with MXFP8 activations) and why it is a stability topic.

Details The report's table gives the FP8 scheme as "tensor-wise, delayed scaling E4M3 / E5M2" for Base-1 and "MXFP8 E4M3" for Base-2; the report says nothing more about numerics, so the rest of this slide is background.

Eight-bit floats. An FP8 number has a sign bit and 7 bits split between exponent and mantissa. E4M3 (4 exponent bits, 3 mantissa bits) spans about 2^{-9} to 448 with a relative resolution of 2^{-3}, one part in eight; E5M2 spans up to 57,344 with a resolution of one part in four. Neither range is wide enough to hold a whole tensor of activations or gradients, whose values span many orders of magnitude, so FP8 training multiplies each tensor by a scale factor to bring it into range before casting, and divides it out after the matrix multiply, which accumulates in higher precision. The two formats exist because of this trade: weights and activations are better served by precision (E4M3), gradients by range (E5M2), in the original NVIDIA recipe that Base-1 followed.

Tensor-wise delayed scaling. With one scale per tensor, the scale must be set by the tensor's largest magnitude; computing that maximum before every cast costs a pass over the tensor, so the common "delayed scaling" recipe uses the maximum from recent steps instead. Two failure modes follow, both of which the slide names. The scale is set by the single largest value, so an outlier squeezes every other value toward zero and into the format's coarse low range. And the delayed scale can be wrong for a step: if a value appears that is larger than the recent history, it overflows to infinity or saturates.

Microscaling. The MX formats (Rouhani et al. 2023, standardized by the Open Compute Project) replace the per-tensor scale with one scale per block of 32 consecutive values, stored as an E8M0 number (an 8-bit exponent, a pure power of two, no mantissa). Each block carries its own range, so an outlier affects only its 31 neighbors, and the scale is computed from the block itself at cast time, so there is no delay. The storage cost is $8/32 = 0.25$ extra bits per value: MXFP8 is 8.25 bits per number. Because range is handled per block, E4M3, the precise format, can be used for gradients as well; the report's table says "E4M3" for Base-2 without qualification. Blackwell tensor cores take MX formats natively, applying the block scales inside the matrix multiply, which is why the format is affordable on GB200 and was not on Hopper. NVIDIA's MXFP8 recipe (2025) reports matching BF16 on an 8B model over 15T tokens and notes one subtlety, that the block scale should be rounded up, not to nearest, so that no value in the block saturates.

Where it meets the rest of the deck. Weight quantization appears in Figure 6 (slide 22) as a compute-stream task overlapped with the exposed dispatch: the expert weights are cast to MXFP8 each step before the expert GEMMs. Kimi K3's model card, per slide 5b, ships 4-bit microscaled weights (MXFP4), the 4-bit member of the same family, with MXFP8 activations.

Terms

FP8 Eight-bit floating point; two variants, E4M3 and E5M2 (Micikevicius et al. 2022).

E4M3, E5M2, E8M0 Exponent/mantissa bit counts. E4M3: max 448, 3 mantissa bits. E5M2: max

57,344, 2 mantissa bits. E8M0: 8 exponent bits, no mantissa, a power of two; used as a block scale.

scale factor A multiplier that brings a tensor (or block) into a format's range before casting; stored alongside and divided out after.

delayed scaling Setting the scale from the maxima of previous steps rather than the current tensor, to avoid an extra pass.

overflow, saturation What happens when a scaled value exceeds the format's maximum: infinity, or clamping to the maximum.

microscaling (MX) Formats in which blocks of 32 values share one E8M0 scale (Rouhani et al. 2023; OCP MX specification).

MXFP8, MXFP4 Microscaling with 8-bit (E4M3) and 4-bit (E2M1) elements; 8.25 and 4.25 bits per value including the scale.

outlier A value far larger than its neighbors; the enemy of any shared scale.

Intuition

One thermostat for the whole building, set from yesterday's weather, versus one per room, set now. The first leaves most rooms cold whenever one room is hot, and fails outright on a day unlike yesterday. Per-block, on-the-spot scales are the second design, and they are what lets an 8-bit format with three mantissa bits carry a 38-day run without a spike.

9.2 Slide 27: Background: global versus activation gradient clipping

Outlier gradients usually come from a few examples in a batch. Global clipping punishes the whole batch for them.

What the slide shows Three bar charts of per-example gradient contributions in one batch of eight, one bar (the fourth) much taller than the rest. Left, the raw batch: one malformed example dominates the norm. Middle, global norm clipping, $g \leftarrow g \cdot \min(1, c/\|g\|)$: every bar is scaled down, the well-behaved ones included. Right, activation gradient clipping: only the outlier's bar is cut, at a dashed threshold c. Two panels: background (global-norm clipping per Pascanu et al. 2013, which Base-1 used alone; weight gradients are sums over the batch, so one outlier sets the norm and the rest pay) and what MAI-Base-2 does (keeps global clipping and adds clipping of activation gradients at selected places; those are per token, so the clip lands only on the offending positions; with Muon this tightened the gradient norm; the clip events double as a data-quality detector).

Details The report says gradient clipping is a common method to stabilize training by reducing the impact of outlier gradients; standard clipping operates on the global norm of the weight gradient for a whole batch; but outlier gradients are typically driven by a small portion of the data within a batch, so an outlier can cause the gradients from the entire batch to be scaled down, including contributions from well-behaved examples. As a more targeted mitigation, it clips the gradients of activations at selected places in the network. Together with Muon, it says, this keeps MAI-Base-2's gradient norm in a much tighter range than MAI-Base-1's (Figure 7).

Global-norm clipping. The weight gradient g is the sum (or mean) over the batch of per-example gradients. Global-norm clipping (Pascanu et al. 2013) rescales the whole vector when its norm exceeds a threshold c:

$$g \leftarrow g \cdot \min\bigl(1, c/\|g\|_2\bigr), \qquad \|g\|_2 \text{ over all parameters jointly.}$$

It guarantees no step is longer than ηc, which is the protection against exploding gradients it was designed for. Its weakness is that it acts after the sum: if one example in a batch of thousands produces a gradient a hundred times larger than the others, the norm is dominated by that example, the factor $c/\|g\|$ is small, and the thousand good examples' contributions are shrunk along with the bad one. The batch's useful signal is thrown away to tame one sequence.

Activation gradient clipping. During backpropagation, the gradient with respect to a layer's activations is a tensor with one row per token. Clipping it, elementwise or per token, at selected layers caps the contribution any single token can make to everything upstream of that point, before the sum over the batch is taken. The good examples pass unchanged; only the token whose backward signal is extreme is cut. The report does not say where the selected places are, what the thresholds are, or whether the clip is per element or per token norm; the deck's picture is the per-token reading.

Why this changes the gradient norm. With only global clipping, a run with frequent outliers has a weight-gradient norm that frequently exceeds c and gets scaled, so the effective step keeps changing size; Figure 7 (next slide) shows Base-1's norm spanning two decades. Removing outliers at the source makes the summed gradient's norm reflect the bulk of the data, which is what the optimizer's learning rate was tuned for. Zhang et al. (2019) give a theory of why clipping speeds up training when the loss's curvature grows with the gradient; the report's contribution is where to clip.

The side effect. A token whose activation gradient is clipped is, by construction, one the model found extremely surprising; the report says it uses the clip events to find malformed training data (Figure 9, slide 29).

Terms

gradient clipping Limiting the size of a gradient before the optimizer step.

global norm The ℓ_2 norm of the concatenation of all parameters' gradients.

clipping threshold (c) The norm (or value) above which clipping rescales; usually 1.0 for global-norm clipping in LLM training.

per-example gradient The gradient a single training sequence would produce alone; the batch gradient is their sum.

activation gradient The gradient of the loss with respect to a layer's output activations, computed during backpropagation; one row per token.

outlier example A training sequence that produces an abnormally large gradient; often malformed data.

exploding gradients Gradients growing without bound through a deep or recurrent network; the problem clipping was invented for.

Intuition

One student in a class of a thousand screams; global clipping lowers the whole room's volume until the scream is tolerable, and nobody else is heard. Activation clipping puts a hand over the screamer's mouth. The volume knob was invented for a different problem (feedback howl), and the report's point is that at this scale the typical outlier is one bad example, not a runaway network.

9.3 Slide 28: Figure 7: gradient norm, per step and rolling median

Log scale. MAI-Base-1's norm spans two decades with spikes; MAI-Base-2's stays in a narrow band for the whole run.

What the slide shows The report's Figure 7: two panels, MAI-Base-1 and MAI-Base-2, fraction of training completed on the horizontal axis and gradient norm on a logarithmic vertical axis from 0.01 to 10. Each panel has a light per-step trace and a dark rolling median over 1,000 steps. Base-1's per-step trace fills a band from about 0.02 to above 1, with spikes reaching 1 to 3 throughout the run; its median falls quickly from about 1 to about 0.02 and stays there. Base-2's per-step trace stays in a band roughly a decade wide, from about 0.02 to 0.1, with its median drifting slowly upward from about 0.02 to a bit under 0.1 by the end. The slide's three readings and a panel on why the gradient norm is the quantity to watch.

Details The report says MAI-Base-2's recipe changes, including activation gradient clipping and Muon, led to a much tighter range of training gradient norm. The figure's legend gives the two series, per step and rolling median over 1,000 steps.

Reading Base-1. The median at about 0.02 is the typical step; the per-step values at 1 to 3 are fifty to a hundred and fifty times the median (derived from the plot). With a global clipping threshold of order 1 (the report does not give Base-1's threshold), every one of those spikes was a clipped step, and the density of the light band shows they were frequent for the whole run, not just at the start. Base-1's loss curve in Figure 5 (slide 18) spiked only early; the gradient norm shows the underlying instability never went away, it was being contained by clipping.

Reading Base-2. The band is about a decade wide instead of two, and nothing reaches 1. The median's slow rise from 0.02 toward 0.1 across the run is visible and the report does not comment on it; a rising gradient norm late in training is commonly seen when the learning rate decays (the gradient norm tends to grow as the weights settle into a sharper region), and Base-2's loss curve does steepen late (slide 18), but this is a reading, not a report statement.

Attribution. The report credits the tighter range jointly to activation gradient clipping and to Muon; the deck adds that a norm that stays predictable is what lets a 38-day run proceed without a rollback. Muon's contribution is plausible on its own terms: an orthogonalized update has a fixed size regardless of the gradient's magnitude, so the optimizer is less sensitive to the gradient norm than Adam is, and the norm it produces is steadier.

Why watch the norm. The learning rate was tuned (slide 17) on the assumption that the optimizer sees steps of a characteristic size. Global clipping fires whenever the norm exceeds its threshold, so a spiky norm means many silently shortened steps and an effective learning rate that the laws did not model; a norm in band means the optimizer sees the step it was tuned for. The gradient norm is also the earliest warning of a divergence: it rises before the loss does.

Terms

gradient norm $\|g\|_2$ of the full weight gradient at a step; the standard health signal of a training run.

rolling median The median over a sliding window (here 1,000 steps); robust to spikes, so it shows the typical level.

per-step trace The raw value at every step; shows the spikes the median hides.

decade A factor of ten on a log axis.

effective learning rate The step size actually taken after clipping; smaller than the nominal rate whenever clipping fires.

Intuition

Figure 5 was the patient's symptoms; Figure 7 is the blood test. Base-1 looked healthy after the first spikes but its gradient norm shows a chronic condition, held in check by clipping every few steps. Base-2's norm is a flat band, and a flat band is what "stable" means to the optimizer: the step it takes is the step it was told to take.

9.4 Slide 29: Figures 8 and 9: a gain that grows with scale, and a bug it found

Clipping is unusual twice over: its efficiency gain rises with model size, and its clip events point at bad data.

What the slide shows Left, the report's Figure 8: held-out efficiency gain over baseline in percent (vertical, 0 to 30) against training compute in FLOPs (horizontal, log, about $10^{21.5}$ to $10^{23.5}$), with five points connected by a line, L18 through L42, rising from about 2% to a labelled +28%. Right, the report's Figure 9: a training sequence from the code data, an Apache license header that stops mid-sentence at "distributed under" and is followed by "# PR Title", "[pull] master from gradle:master", "# PR Body", "See"; the # is highlighted as the token whose activation gradient was clipped. The slide's readings of each figure and the lesson.

Details Figure 8. The report says activation gradient clipping also made optimization more stable, and its benefit grew with scale as shown in Figure 8, whereas most changes it has tested yield a roughly constant efficiency gain across scales. The figure's vertical axis is $\mathrm{EG}_{\mathrm{FLOPs}}$ on held-out evaluations (slide 19), expressed as a percentage gain; only the top point is labelled, +28% at L42. Reading the others off the plot: about +2% at L18, +7% at L24, +9–10% at L30 and +16% at L36, at compute from roughly 2×10^{21} to 2×10^{23} FLOPs. The curve is convex on the log-compute axis: the gain is growing faster than linearly in $\log C$ over the ladder's range.

Why a growing gain is unusual, and what it implies. A typical recipe change (a better activation function, a tweak to the normalization) gives roughly the same percentage saving at every rung, so its value at the final scale can be read from any rung. A change whose gain grows with scale is both more valuable and harder to evaluate: a test at L18 would have rated clipping at 2% and dismissed it. The ladder's range of two decades is what made the trend visible, and the trend, not any single point, is the argument for adopting it. Extrapolating the curve two more decades to $10^{25.4}$ is tempting but the report does not do it, and the notes will not either; the honest statement is that the gain at MAI-Base-2's scale is at least the 28% measured at L42 if the trend merely held flat, and larger if it continued.

A mechanism consistent with the trend: larger models are trained with smaller learning rates and are more sensitive to outlier gradients (Wortsman et al. 2023 find the stable learning rate falls with size), so removing outliers buys more at scale; and the data was the same at every rung, so the *fraction* of outlier sequences was constant while their damage grew.

Figure 9. The report says it also uses activation gradient clipping as a diagnostic tool for validating training data; Figure 9 shows a malformed training example identified by it, a pull request from the code data in which a license header is abruptly cut off mid-sentence; tracing the clipped example back revealed a bug in the preprocessing pipeline, which has since been fixed. The caption identifies the highlighted # as the token whose activation gradient was clipped: the first token after the truncation, where the model, having predicted the rest of a license it has seen millions of times, meets something it could not have expected. The gradient at that token is extreme precisely because the model's prediction was confident and wrong.

The lesson. A sequence that forces an extreme gradient is often malformed, so the log of clipped tokens is a free data audit that points at exactly the positions where the corpus contradicts the model's well-founded expectations. This closes a loop in the report: the data pipeline (Section 10) is validated by the training run, not only by its own filters.

Terms

efficiency gain by rung EG_{FLOPs} of one change measured at each ladder size; its trend across rungs is what Figure 8 plots.

scale-dependent gain A change whose benefit changes with model size; requires a ladder with range to detect.

malformed example A training sequence corrupted by preprocessing (truncation, concatenation, encoding errors); a common source of outlier gradients.

clip event A step and position at which activation gradient clipping fired; logged, it indexes suspicious data.

preprocessing pipeline The code that turns raw crawled documents into training sequences; Figure 9's bug lived there.

Intuition

Two surprises from one mechanism. First, a change that looked like a 2% trick at the bottom of the ladder was a 28% win at the top, which is the strongest argument in the report for running ladders with range rather than one pilot. Second, the clipper is also a smoke detector: wherever it fires, something in the data was not what the model had every right to expect. Half a license header and a stray # found a pipeline bug that no filter had caught.

10 Data (slides 30–32)

10.1 Slide 30: Scaling data: every token processed in-house

Organic data with provenance, no distillation from other models, and a pipeline that was rebuilt between the two models.

What the slide shows A pipeline of eight stages: crawl (robots.txt) → parse → quality classify → topic bin → dedup → diversity sample → decontaminate → mix, with notes under five of them: respects robots.txt; improved since Thinking-1; by embedding, prefer novel; against all eval sets; Figure 10. Below: the principles (organic data, no distillation; every token processed in-house; provenance good enough to license the weights through Foundry) and what is new for Base-2 (much more web content, a March 2026 cutoff, embedding-based diversity sampling; exclusions for Responsible AI policy and the USTR Notorious Markets list). A background panel on diversity by embedding: SemDeDup (Abbas et al. 2023) and D4 (Tirumala et al. 2023).

Details The report says MAI-Base-2 follows the same data principles as MAI-Base-1: its path to frontier performance is built on organic data, not distillation from other models; it processed every training token in-house; and its corpus grew by roughly 36% from Base-1 to Base-2, measured in unique training tokens. Since MAI-Thinking-1 it improved its quality classification, topic binning, web parsing, deduplication and evaluation decontamination; for Base-2 it expanded its pipelines to process substantially more web content, with a March 2026 knowledge cutoff; and it used embedding-based diversity sampling to prioritize novel data and reduce duplicative content, which its experiments showed substantially improved performance. A footnote says the web crawler respects the Robots Exclusion Protocol and related meta-tag and HTML controls, and that sources violating Microsoft Responsible AI policies or on the USTR Notorious Markets list are excluded. Elsewhere the report says strong data provenance is what will enable licensing the weights to partners through Foundry.

The eight stages on the slide are the deck's arrangement of the operations the report names; the report does not draw a pipeline or give an order. Each stage is a standard component of a modern pre-training data pipeline (FineWeb and DataComp-LM document open versions): a crawler fetches pages; a parser extracts text from HTML; quality classifiers score documents (DataComp-LM finds this the single largest lever); topic binning groups documents for mixture control; deduplication removes exact and near-exact repeats (Lee et al. 2021 showed this alone improves models); decontamination removes overlaps with evaluation sets; and mixing sets the proportions of Figure 10.

Diversity by embedding. Exact and hash-based deduplication catch copies; they miss the vast number of documents that say the same thing in slightly different words. SemDeDup (Abbas et al. 2023) embeds documents with a pretrained encoder, clusters them, and removes near-duplicates within each cluster, finding that a large fraction of web data can be removed with no loss. D4 (Tirumala et al. 2023) adds a diversification step that selects, from the deduplicated set, documents far from the already-chosen ones in embedding space, and reports that training on the selected subset beats random selection at equal tokens. "Prioritize novel data" in the report is this idea; the implementation is not described. Diversity sampling and source scaling interact: a corpus with less hidden repetition behaves more like its unique token count, which is the number source scaling is built on.

Why "organic" and "in-house" are emphasized. Distillation, training on another model's outputs, is a fast way to raise benchmark scores and also a way to inherit another model's errors and license terms; the report's position is that a base model built only on primary data is both more defensible and, by Figure 3, more efficient per FLOP. Provenance over every token is a legal property (needed to license weights) and an engineering one (needed for Figure 9's trace from a clipped token back to a pipeline bug).

Terms

organic data Text and images produced by people in the ordinary course of things, as opposed to data generated by a model.

distillation Training a model on the outputs of another model; the report rules it out for pre-training.

provenance A record of where every training token came from and under what terms.

robots.txt The Robots Exclusion Protocol: a file on a web site that states which crawlers may fetch which pages.

quality classifier A model that scores documents for usefulness as training data; used to filter or weight the web.

topic binning Assigning documents to subject categories so that the mixture can be set per topic.

deduplication Removing repeated documents or passages, exact (hashing) or near (embedding).

SemDeDup, D4 Embedding-based near-duplicate removal (Abbas et al. 2023) and diversity-based selection (Tirumala et al. 2023).

knowledge cutoff The date after which no training data was collected; March 2026 for Base-2.

Foundry Microsoft's platform through which, the report says, the model weights will be licensed to partners.

Intuition

The model is what it eats, and the report's kitchen rules are: cook from raw ingredients, no leftovers from other kitchens, know every supplier, and vary the menu. The last rule is the technical novelty for Base-2: choosing documents for what they add rather than for what they are, with an embedding as the measure

of novelty.

10.2 Slide 31: Figure 10: the mixture, 30T tokens to 50T

Solid segments are unique tokens, light segments are repeats. Vision data is new; books and PDFs grew most.

What the slide shows The report's Figure 10: two stacked horizontal bars. The top bar, MAI-Base-1, is 30T tokens with segment shares bracketed above it: 55% code, 15% web, 5% PDFs, 16% STEM, 3% books, 5% math, 2% other. The bottom bar, MAI-Base-2, is 50T tokens with shares labelled below: code 45%, web 12%, PDFs 9%, STEM 9%, books 5%, math 4%, vision-only 14%, other 2%. Each segment is split into a solid part (unique tokens) and a lighter part (repeated tokens), and ribbons connect each Base-1 source to its Base-2 counterpart with a growth label: +34% code, +16% web, +104% PDFs, +24% STEM, +194% books, +37% math, +22% other. The slide's table turns the Base-2 shares into token counts and lists the growth; its text covers unique tokens, code, and vision.

Details The report says its corpus grew by roughly 36% in unique training tokens from Base-1 to Base-2; that the VLM corpus contains 4.5T unique tokens and, with repetition, VLM data accounted for 18% of the 50T-token mixture; and, in the figure caption, that solid segments are unique tokens and light segments repeated tokens.

Token counts (derived from the 50T total and the shares read off the figure): code 22.5T, vision-only 7T, web 6T, PDFs 4.5T, STEM 4.5T, books 2.5T, math 2T, other 1T. Code fell from 55% to 45% of the mixture but grew in absolute terms, from 16.5T to 22.5T of the (repeated) total (derived).

What the growth labels measure. The percentages on the ribbons cannot be reproduced from the shares and totals: web at 15% of 30T is 4.5T and at 12% of 50T is 6T, a 33% increase, not 16%; STEM at 16% of 30T is 4.8T and at 9% of 50T is 4.5T, a decrease, not +24%. So the labels must refer to the solid (unique) segments, consistent with the report's own measure of corpus growth being unique tokens; on that reading the unique data in every source grew, books by nearly a factor of three and PDFs by a factor of two, and the overall 36% is the token-weighted average. The slide's table should be read with this in mind: shares are of the repeated mixture, growth is of unique tokens.

Vision. The report's 4.5T unique VLM tokens and 18% of 50T, which is 9T tokens, give two passes over the vision corpus on average (derived). The 14% vision-only segment (7T) is part of the 18%; the remaining 4% is presumably interleaved image-text documents counted under other sources, which the report does not break out. The vision data is the largest single difference between the two mixtures and the reason the slide after this one exists.

Repetition in the mixture. The visible split between solid and light segments is the data behind source scaling (slide 12): sources differ in how many times they are repeated, with the small, high-value sources (books, math, STEM) repeated more than the web. Muennighoff et al.'s result that up to about four passes are nearly free is what licenses this, and the per-source repeat counts are what the ladder's lower rungs reproduce.

Terms

mixture The proportions of each source in the training stream; Figure 10's shares.

repeated tokens Tokens seen more than once because a source is cycled; the light segments.

vision-only data Image data without paired text in the same document; 14% of the mixture.

VLM corpus All vision-language data, image-text pairs and interleaved documents; 4.5T unique tokens, 18% of the mixture with repetition.

PDFs, books, STEM The document-type sources whose unique data grew most; PDFs are parsed documents from the web, STEM is scientific and technical text.

Intuition

Two bars tell a quiet story: the corpus got a third bigger, images arrived, and the model still spends almost half its time on code. The solid-versus-light split is the honest part of the figure; it shows how much of a 50-trillion-token run is re-reading, and which shelves get re-read most.

10.3 Slide 32: Native vision without an image encoder

MAI-Base-2 is pre-trained on images from the start, with patches fed into the same transformer as the text.

What the slide shows Two block diagrams. The common design: image → vision encoder → projector → transformer, with text tokens entering the transformer separately; annotated as pre-trained separately (for example a CLIP-style ViT), usually added after text pre-training, with a fixed input resolution. MAI-Base-2's design: image → patches → linear embed → transformer, the same transformer the text tokens enter; annotated as one model from the first token, vision and text sharing every parameter, needing far more vision data, hence 4.5T unique VLM tokens; precedents Fuyu-8B (Adept 2023) and EVE (Diao et al. 2024). The footer lists the corpus's task families per the report.

Details The report says that unlike MAI-Base-1, MAI-Base-2 is pre-trained natively with vision using an encoder-free VLM architecture; that it invested heavily in growing its VLM corpus across image-text pairs and documents with interleaved text and images; and that the corpus spans image captioning from the web, chart understanding, visual grounding, spatial reasoning, medical understanding, computer use, sheet-music reading and other areas. The architecture row of its table says "Native VLM". Nothing more is said about the vision architecture; the diagrams are the deck's rendering of what "encoder-free" means in the literature.

The encoder-based design. Most vision-language models of 2023–25 (LLaVA, PaliGemma; Llama 3's vision models use a cross-attention variant) take a vision transformer pre-trained separately on image-text contrastive data (CLIP-style), run the image through it to get a few hundred feature vectors, map those into the language model's embedding space with a small projector, and insert them into the token sequence. The language model is usually fully pre-trained on text first, and the vision parts are attached and tuned afterwards. The advantages are economy (the encoder has already learned vision from its own data) and modularity; the costs are a fixed input resolution set by the encoder, a bottleneck at the projector, and a model that learned to see and to read separately.

The encoder-free design. Fuyu-8B (Adept 2023) showed that a decoder-only transformer can take image patches directly: the image is cut into patches, each patch's pixels are linearly projected to a token embedding, and the resulting tokens are placed in the sequence like text tokens, with no encoder and no projector; any resolution works because the number of patch tokens simply varies. EVE (Diao et al. 2024) studied what it takes to make this match encoder-based models and found the answer is mostly data and training time: the transformer has to learn vision from scratch. "Natively" in the report means that this learning happened during pre-training, from the first token, on the 4.5T-token vision corpus, rather than being added to a finished text model.

Why it fits the rest of the report. Native vision is the data-side counterpart of "simple methods scale best": one architecture, one training run, no separately pre-trained component with its own provenance and license. The price is paid in data, which the report says it invested heavily in, and the task list in the corpus description (charts, grounding, spatial reasoning, computer use, sheet music) reads as a list of what a text-only base model cannot do and a product needs.

Terms

vision-language model (VLM) A model that takes images and text and produces text.

vision encoder A separately trained network (usually a vision transformer) that turns an image into

feature vectors; absent in an encoder-free design.

projector A small network mapping encoder features into the language model's token-embedding space.

patch A small square of pixels (16×16 or similar) treated as one token after linear embedding (ViT, Dosovitskiy et al. 2020).

encoder-free VLM A design in which patch embeddings enter the language transformer directly (Fuyu-8B; EVE).

native multimodal pre-training Training on images and text together from the start of pre-training, rather than adding vision afterwards.

interleaved documents Web pages and documents in which images and text alternate in reading order; a source of VLM training data.

visual grounding Linking a phrase to the image region it refers to.

Intuition

The common design hires a trained photographer and asks the writer to work from the photographer's notes. The encoder-free design teaches the writer to see, which takes far more pictures but ends with one person who does both. The report's bet, consistent with the rest of it, is that data can replace architecture, and that the extra 4.5 trillion tokens were cheaper than the seams.

11 Wrap-up (slides 33–34)

11.1 Slide 33: Part One's components, Part Two's choices

The same block diagram, filled in with what MAI-Base-2 actually does, and the three principles the report closes on.

What the slide shows A two-column table that walks down Part One's components and states MAI-Base-2's choice for each: tokens, embedding, unembedding (untied input and output tables); attention (96 query heads; 2 K/V heads in global layers, one in six, 16 in local layers); MLP (a mixture of 512 experts, 1 shared + 7 routed per token, in every layer); block and stack (102 blocks, width 8,704; 2.8T parameters total, 80B active); output and loss (bits per byte on held-out code, QA, STEM and math); training (50T tokens; Muon + AdamW; MXFP8 E4M3; weight and activation gradient clipping); hyperparameters (from fitted laws: LR, weight decay, dropout as functions of steps, TPP and size); scale (16k GB200 GPUs, 38 days, 21% MFU, 89% goodput, $\approx 2.4 \times 10^{25}$ FLOPs). A note lists what the report does not state: positional encoding, normalization placement, vocabulary size, context length. Three cards give the report's principles: learned from first principles (organic data, scaling science, no distillation); simple methods scale best (one ladder, one recipe, one metric); rigor over shortcuts (laws fitted, validated, then trusted).

Details Every entry in the table is a report fact already discussed (slides 3, 6, 7, 8, 17, 26, 27) except the FLOPs, which are derived. The value of the table is the mapping: each box of Part One's diagram has become a design decision with a stated reason and, usually, a figure behind it.

What is not in the report. Four things a student of Part One will look for are absent: the positional encoding (RoPE or otherwise), where the normalization layers sit (pre-norm, post-norm, QK-norm), the vocabulary size and tokenizer, and the context length (and the local window). The absence is itself informative about what the report is for: it is an account of method (ladders, laws, co-design, data) rather than a model card, and the authors chose to document the parts of the recipe they consider their contribution. The notes have

flagged every place where a number was derived under an assumption about one of these (head dimension on slide 10, vocabulary on slide 8).

The three principles, as the report states them: intelligence should be learned from first principles; simple methods scale best; scientific rigor over shortcuts. The deck's gloss on each card ties it to the body of the report. First principles: organic data and scaling science rather than distillation and imitation. Simple methods: one ladder with one rule, one recipe for every rung, one metric (BPB, or EG for changes); plain attention, plain softmax routing, the oldest form of local attention. Rigor: hyperparameter laws fitted at small scale, validated at an intermediate scale against the old heuristics, then applied without a sweep at the final scale; a forecast made before the run and checked after.

Terms

positional encoding How the model learns token order (RoPE, learned positions, none); not stated for MAI-Base-2.

normalization placement Whether layer norms precede or follow each sublayer, and whether queries and keys are normalized; not stated.

vocabulary size (V) Number of distinct tokens the tokenizer produces; not stated.

context length The maximum sequence length the model is trained on; not stated.

first principles The report's phrase for learning from primary data and from experiments rather than from other models.

Intuition

Read the table top to bottom as the answer key to Part One. For each component, the question "what would a frontier lab choose, and how would it know?" now has an answer and a method. The four blanks at the bottom are a reminder that even a thirteen-page report with ten figures leaves the recipe incomplete.

11.2 Slide 34: Sources

All MAI-Base-2 facts and figures are from the report; background references were checked against their arXiv pages.

What the slide shows Two columns of references grouped by topic: the report and the comparators' model cards; mixture of experts; scaling; hyperparameters; optimizer and systems; numerics and stability; attention, embeddings, vision and data. Each background reference carries its arXiv identifier or URL.

Details The deck's sourcing rules, which these notes follow: every MAI-Base-2 fact is from the report (Microsoft AI, *Frontier Pre-training at Microsoft AI*, microsoft.ai/news, 15 October 2026, PDF version 2), and Figures 1–10 are reproduced from it; comparator facts are from the models' Hugging Face model cards and configuration files as of October 2026 (deepseek-ai/DeepSeek-V4-Pro-Base, moonshotai/Kimi-K3, Qwen/Qwen3.8-2.4T-A95B, XiaomiMiMo/MiMo-V2.6-Pro-RL); background claims cite the paper that made them. Appendix D expands the slide into a reading list by section, with one line on why each paper is worth reading, and matches the paper archive that accompanies these notes.

Three reading suggestions for a student who wants to go one level deeper than the deck. For the ladder method, Bhagia et al. (2024) and Choshen et al. (2024) together cover how ladders are built and how they fail. For Muon, Jordan's blog post is short and complete, and Bernstein and Newhouse (2024) explain why it works. For the MoE systems section, MegaBlocks (Gale et al. 2022) is the paper against which the report's multi-round design should be read.

Terms

arXiv identifier The YYMM.NNNNN number that locates a preprint at arxiv.org/abs/; the key used throughout the reading list.

secondary source A model card, blog post or vendor page rather than a paper; marked as such in the reading list.

Intuition

A deck about a report is only as good as its sourcing. Everything the slides say about MAI-Base-2 can be checked against thirteen pages; everything else can be checked against a preprint. Where the notes derived a number, they say so; where the report is silent, they say that too.

A Notation

symbol	meaning	notes
N_{total}	total parameters	everything stored; 2.8T for Base-2
N_{active}	active parameters	used by one token; 80B for Base-2; sets FLOPs
D	training tokens	50T for Base-2; 30T for Base-1
C	training compute, FLOPs	$\approx 6 N_{\text{active}} D$
L	number of layers	names each ladder rung (L18 … L102)
d	hidden dimension	$(256/3)L$ on the ladder; 8,704 for Base-2
h, h_{kv}, d_h	query heads, KV heads, head dimension	96; 2 (global) or 16 (local); d_h not stated
E, k	routed experts, active per token	512, 7 (+1 shared)
g_i	router score of expert i	weights the expert's output
T, CF, C_{cap}	tokens per batch, capacity factor, capacity	$C_{\text{cap}} = \text{CF} \cdot kT/E$ per expert
TPP	tokens per active parameter	D/N_{active}; 625 for Base-2 (677 in Figure 4)
BPB	bits per byte	$\frac{1}{B}\sum_i -\log_2 p(x_i \mid x_{<i})$
$\overline{\text{NLL}}$	mean negative log-likelihood	nats per token
N, B	tokens, bytes in a held-out text	N/B is the tokenizer's tokens per byte
$\mathscr{L}$	loss	$E + AC^{-\alpha}$ or Chinchilla's $E + A/N^{\alpha} + B/D^{\beta}$
η, λ	learning rate, weight decay	LR, WD in the report; $\eta\lambda$ sets the EMA timescale
τ	EMA timescale of the weights	$1/(\eta\lambda)$ steps $=$ steps$/17.04$
EG	efficiency gain	$C_{\text{baseline}}/C_{\text{candidate}}$ at equal loss; in FLOPs or time
M, O	momentum matrix, its orthogonalization	$O \approx UV^{\top}$ for $M = U\Sigma V^{\top}$
X_k	Newton–Schulz iterate	$X_0 = M/\|M\|_F$
a, b, c	Newton–Schulz coefficients	Jordan's 3.4445, -4.7750, 2.0315
g, c	weight gradient, clipping threshold	$g \leftarrow g\min(1, c/\|g\|)$
EeMm	float format with e exponent, m mantissa bits	E4M3, E5M2, E8M0
s	FP8 / MX scale factor	per tensor (Base-1) or per block of 32 (Base-2)

B Formulas at a glance

Cost model. training FLOPs $C \approx 6 N_{\text{active}} D$ (2 forward, 4 backward per active parameter per token); MAI-Base-2: $6 \times 80\text{B} \times 50\text{T} = 2.4 \times 10^{25}$. Hardware side: $C \approx n_{\text{GPU}} \times \text{peak} \times \text{time} \times \text{MFU} \times \text{goodput}$; $16{,}000 \times 2.5 \times 10^{15} \times 38 \times 86{,}400 \times 0.21 \times 0.89 \approx 2.45 \times 10^{25}$.

Tokens per parameter. $\text{TPP} = D/N_{\text{active}}$; Chinchilla-optimal ≈ 20; MAI-Base-2 $50\text{T}/80\text{B} = 625$. At fixed C, $D = C/(6N_{\text{active}})$, so $\text{TPP} = C/(6N_{\text{active}}^2)$.

Bits per byte. $\text{BPB} = \dfrac{1}{B}\sum_{i=1}^{N} -\log_2 p(x_i \mid x_{<i}) = \dfrac{N}{B} \cdot \dfrac{\overline{\text{NLL}}}{\ln 2}$, with $\overline{\text{NLL}}$ in nats per token and N/B tokens per byte. Inverse: $\overline{\text{NLL}} = \text{BPB} \cdot \ln 2 \cdot (B/N)$. Compression ratio $= 8/\text{BPB}$ ($0.22 \to 36\times$). Bits per token $= \text{BPB} \cdot B/N$.

Scaling laws. Kaplan/Chinchilla $\mathscr{L}(N,D) = E + A/N^{\alpha} + B/D^{\beta}$ (Chinchilla's fit: $E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$, $\beta = 0.28$); against compute $\mathscr{L}(C) = E + AC^{-\alpha}$. Equal-quality compute ratio (Figure 3): solve $\mathscr{L}(C^*) = \text{BPB}_{\text{ext}}$ on the ladder's fit, ratio $= C_{\text{ext}}/C^*$.

Scaling ladder. $d = \frac{256}{3}L$; query heads $= L$ rounded to a multiple of 16; TPP fixed; repetition per dataset fixed (source scaling: a rung with $1/r$ of the tokens trains on $1/r$ of the unique data, same number of passes). Check: $L = 102 \Rightarrow d = 8{,}704$, 96 heads.

Hyperparameter laws (report). $\text{LR}^* \times \text{WD}^* = 17.04/\text{steps}$;
$\log \text{LR}^* = -1.51 + 0.175 \log \text{TPP} - 0.573 \log \text{steps}$ (natural logs);
$\text{Dropout} = \text{sigmoid}\big(0.526\,(\log \text{ActiveParams} - 27.2)\big)$.
EMA timescale $\tau = 1/(\text{LR} \cdot \text{WD}) = \text{steps}/17.04 \approx 5.9\%$ of the run. Square-root rule: at fixed tokens, $\text{LR} \propto \text{batch}^{1/2} \propto \text{steps}^{-1/2}$.

AdamW step. $\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\, u_t$; unrolled, an EMA of updates with timescale $1/(\eta\lambda)$.

Efficiency gain. $\text{EG} = C_{\text{baseline}}(\mathscr{L})/C_{\text{candidate}}(\mathscr{L})$ at equal loss, in FLOPs or wall-clock time; gain $= \text{EG} - 1$. Relation: $\text{EG}_{\text{time}} \approx \text{EG}_{\text{FLOPs}} \times (\text{throughput}_{\text{candidate}}/\text{throughput}_{\text{baseline}})$.

MoE layer. $y = x + \sum_{i \in \text{Top}k(g)} g_i E_i(x) + E_{\text{shared}}(x)$; expected tokens per expert kT/E; capacity $C_{\text{cap}} = \text{CF} \cdot kT/E$; with token dropping, tokens beyond C_{cap} skip the layer; multi-round dropless: rounds $= \lceil \max_i(\text{load}_i)/C_{\text{cap}} \rceil$ per group (fewer with intra-group borrowing), memory fixed at C_{cap} per expert.

MoE throughput. MoE-layer MFU $24\% \to 35\%$ is $1.46\times$ the throughput at equal FLOPs.

Muon. $M \leftarrow \beta M + G$; $X_0 = M/\|M\|_F$; $X_{k+1} = aX_k + b(X_kX_k^{\top})X_k + c(X_kX_k^{\top})^2X_k$ five times; $O = X_5 \approx UV^{\top}$; $W \leftarrow W - \eta\, s\, O$. On singular values the iteration is $\sigma \mapsto a\sigma + b\sigma^3 + c\sigma^5$. Cost: 2.1% of an iteration $= 165$ ms $\Rightarrow$ iteration ≈ 7.9 s (derived); floor 55 ms.

Gradient clipping. global: $g \leftarrow g \min(1, c/\|g\|_2)$ after the batch sum; activation: clip $\partial\mathscr{L}/\partial a$ per token at selected layers before the sum.

FP8 and MXFP8. value $= (-1)^s 2^{e-\text{bias}}(1 + m/2^M)$; E4M3 max 448, E5M2 max 57,344; tensor-wise: $\hat{x} = \text{fp8}(x/s)$ with $s = \max|x|/448$ (delayed: from past steps); MX: one E8M0 scale $s_b = 2^{\lceil \log_2(\max_b |x|/448) \rceil}$ per block of 32, cast $\text{fp8}(x/s_b)$; storage $8 + 8/32 = 8.25$ bits per value (MXFP4: 4.25).

Attention cache. KV bytes per token $= 2 n_{\text{layers}} h_{kv} d_h \cdot \text{bits}/8$ for global layers; local layers hold at most the window w tokens. Base-2: 17 global layers with $h_{kv} = 2$, 85 local with $h_{kv} = 16$.

Worked numbers (derived). MoE-layer speedup $35/24 = 1.46$. Muon iteration $165/0.021 = 7{,}857$ ms. DeepSeek-V4-Pro-Base lower bound $6 \times 49\text{B} \times 32\text{T} = 9.4 \times 10^{24}$ FLOPs; MAI-Base-2 at most $2.5\times$ that. Vision passes $9\text{T}/4.5\text{T} = 2$. Figure 10 token counts: $0.45 \times 50\text{T} = 22.5\text{T}$ code, etc.

C Exercises

1. **Bookkeeping.** Using the report's numbers (80B active, 50T tokens, 16k GB200 GPUs at 2.5 PFLOPS BF16, 38 days, 21% MFU, 89% goodput), compute the training FLOPs by the $6ND$ rule and by the hardware count, and state the ratio. If the systems team raised MFU to 30% at the same goodput, how many days would the same run take? What MFU would a 30-day run have needed? Why does the report's MFU definition (excluding rematerialization and optimizer FLOPs) make the $6ND$ count the right comparison?

2. **Bits per byte.** A model has a mean held-out NLL of 0.62 nats per token with a tokenizer that averages 4.1 bytes per token. Compute its BPB. Conversely, MAI-Base-2 scores 0.2203 BPB on held-out code; if its tokenizer averages 3.6 bytes per token on code (an assumption), what is its mean NLL in nats per token and in bits per token? By what factor would an arithmetic coder driven by it compress the code? Explain why two models with different tokenizers can be compared in BPB but not in NLL per token.

3. **Parameter count (derived, with an assumption).** The report gives 102 layers, 512 routed experts per layer, an MoE down-projection dimension of 4,096 and an expert FFN dimension of 8,704. Suppose each routed expert is a gated MLP on the 4,096-wide projected input whose gate and up projections together span the 8,704 width (4,352 each) and whose down projection maps 4,352 back to 4,096. Compute the parameters per expert, per layer, and for all 102 layers, and compare with 2.8T. Repeat under the alternative reading that gate and up are each 8,704 wide, and explain which reading the total supports. Then estimate the active parameters by adding 7 routed experts, one shared expert of the same inner width on the full 8,704 stream, and attention with 96 query heads of dimension 128 (assumed), 2 or 16 KV heads, and an output projection; compare with 80B and list every assumption.

4. **KV cache.** Assume a head dimension of 128 and a local window of 4,096 tokens (neither is stated in the report). For MAI-Base-2 (17 global layers with 2 KV heads, 85 local layers with 16 KV heads) compute the KV-cache bytes per token for the global layers at BF16, the total cache at a 128k-token context, and the fixed contribution of the local layers. Repeat for MAI-Base-1's layout (78 layers, 8 KV heads everywhere, global every sixth layer) and comment on where the saving comes from.

5. **Ladder rules.** For $L \in \{12, 18, 24, 30, 36, 42, 102\}$ compute the ladder's hidden width $(256/3)L$ and query-head count (L rounded to a multiple of 16; note the tie at $L = 24$). Assuming the non-embedding active parameters of a rung scale as Ld^2 times a constant, and that TPP is fixed, show that training compute scales as L^6 along the ladder, and check against the compute positions of L18 ($\approx 2 \times 10^{21}$) and L42 ($\approx 2 \times 10^{23}$) read off Figure 2. What does the mismatch, if any, tell you about which parameters (attention, experts, embeddings) do not scale as d^2?

6. **Source scaling.** A dataset is repeated 8 times in the 50T-token run. Under source scaling, how many unique tokens of it does a 5T rung see, and a 500B rung? Using Muennighoff et al.'s finding that repeated tokens are nearly as valuable as new ones for up to about 4 passes and decay afterwards, explain why a 5T rung trained without source scaling (5T unique tokens, one pass) would over-estimate the value of a larger expert width, and why it would under-estimate the right dropout rate.

7. **Chinchilla vs. MAI-Base-2.** Using Chinchilla's fit $\mathscr{L} = 1.69 + 406.4/N^{0.34} + 410.7/D^{0.28}$ (with N in parameters and D in tokens), evaluate the predicted loss at $N = 80 \times 10^9$, $D = 50 \times 10^{12}$. Then find the compute-optimal (N, D) for the same compute $C = 6ND$ (minimize $\mathscr{L}$ subject to $6ND = C$) and its loss. How much lower is the compute-optimal loss, and why would a frontier lab accept the gap? (The fit is for dense models on 2022 data; treat the exercise as an order-of-magnitude argument.)

8. **Figure 3 by hand.** Suppose a ladder's held-out BPB against compute fits $\mathscr{L}(C) = 0.19 + 2.6C^{-0.05}$ (invented constants). An external model trained with 10^{25} FLOPs scores 0.237 BPB. Find the compute at which the fitted curve reaches 0.237 and the implied FLOPs-efficiency ratio. Then show that if the external model's compute were mis-stated by a factor of 2, the ratio changes by the same factor, and discuss why the report chose a comparator whose base weights and token count are public.

9. **Hyperparameter laws.** Using the report's laws (natural logs), compute LR^* and WD^* for a run of 100,000 steps at TPP 625, and the EMA timescale in steps. Verify the law against two fit minima from Figure 4: for L12 at TPP 30 ($\mathrm{LR}^* = 0.00374$, $\mathrm{WD}^* = 0.869$) and L12 at TPP 300 ($\mathrm{LR}^* = 0.00197$, $\mathrm{WD}^* = 0.462$), infer the step counts from $\mathrm{LR}^* \times \mathrm{WD}^* = 17.04/\text{steps}$, then check whether the LR law reproduces the two learning rates. What batch-size ratio between the two sweeps do the inferred step counts imply?

10. **Exponents.** From $\mathrm{LR}^* \times \mathrm{WD}^* = 17.04/\text{steps}$ derive that the weight EMA timescale is a fixed fraction of training and compute that fraction. From the LR law, show that at fixed tokens per parameter and fixed tokens, doubling the batch (halving the steps) multiplies LR^* by $2^{0.573}$, and compare with the $\sqrt{2}$ rule. Under the assumption that ActiveParams is in raw parameters and the log is natural, evaluate the dropout law at 35B, 80B and 650B active parameters.

11. **Two clocks.** A candidate change lowers the FLOPs needed to reach the baseline loss by 15% ($\mathrm{EG}_{\mathrm{FLOPs}} = 1/0.85$) but its kernels lower the run's MFU from 21% to 17%. Compute $\mathrm{EG}_{\mathrm{time}}$ and say which quadrant of slide 19 it lands in. What MFU would make it break even on time? Now apply the same reasoning to MoE-every-layer in the Base-1 era, given that the MoE-layer MFU rose from 24% to 35%: if MoE layers were half the step time before the fix, by what factor did the whole step speed up (ignore other changes)?

12. **Capacity and rounds.** An expert-parallel group holds 8 experts and receives, in one batch, $kT/E = 2{,}048$ tokens per expert on average. Model the per-expert load as a Poisson variable with that mean (a crude model; real routing is more skewed). With capacity factor 1.25, what is the probability that a given expert overflows, and that at least one of the 8 does? If overflow is handled by a second round over the whole group, what is the expected number of rounds? Repeat with capacity factor 1.1 and discuss the memory-versus-rounds trade-off the report describes, and how borrowing unused capacity within the group changes the overflow probability.

13. **Newton–Schulz on singular values.** The iteration acts on each singular value as $\sigma \mapsto a\sigma + b\sigma^3 + c\sigma^5$ with Jordan's $a = 3.4445$, $b = -4.7750$, $c = 2.0315$. Starting from $\sigma_0 \in \{0.05, 0.2, 0.5, 1.0\}$ (after the $\|M\|_F$ normalization all singular values lie in $(0, 1]$), compute five iterations and report the final values. Show that $\sigma = 1$ is a fixed point, that the map is not monotone (it overshoots 1 for some inputs) and explain why "all singular values close to 1" rather than "exactly 1" is good enough for an optimizer. Count the matrix multiplies per iteration for an $m \times n$ matrix with $m \le n$.

14. **Muon's budget.** From 165 ms = 2.1% of an iteration, compute the iteration time. With 38 days at 89% goodput, estimate the number of iterations and the tokens per iteration (batch size in tokens), and compare log(steps) with the range 8–12 on Figure 4's horizontal axis. If the all-to-all floor is 55 ms and Muon were brought to it, what fraction of an iteration would it take and how many GPU-days would be saved over the run? (All derived; state the assumption that the average iteration time is constant.)

15. **Block scales.** A block of 32 activations contains one value of magnitude 300 and thirty-one values of magnitude about 0.4. (a) With one E4M3 scale for a whole tensor whose maximum is 3,000, what E4M3 codes do the 0.4 values receive, and what relative error? (b) With an MX block scale $s_b = 2^{\lceil \log_2(300/448) \rceil}$

for this block, what codes do they receive? (c) If a delayed tensor scale was set from a previous step whose maximum was 300, what happens to a value of 3,000 arriving now? (d) Compute the bits per value of MXFP8 and MXFP4 including scales, and the bytes needed to store 2.8T weights in each.

16. **Clipping arithmetic.** A batch of 4,096 sequences has per-sequence gradient vectors of norm about 1 in random directions, except one of norm 150. Approximate the norm of the summed gradient with and without the outlier (random directions add in quadrature). With global clipping at $c = 1$ applied to the mean gradient, what factor is applied in each case, and what fraction of the well-behaved sequences' contribution survives when the outlier is present? Now suppose activation gradient clipping caps the outlier's contribution at norm 3 before the sum; recompute. Then fit a straight line through Figure 8's points (gain in % against $\log_{10} C$: $(21.3, 2)$, $(22.0, 7)$, $(22.5, 9.5)$, $(22.85, 16)$, $(23.3, 28)$, all approximate) and say why extrapolating it to $10^{25.4}$ would be unwise.

D Reading list by section

The papers behind each part of the deck, in the order they are discussed, with one line on why each is worth reading. Entries with an arXiv identifier are included in the paper archive; entries without one are listed with a URL. Items marked *secondary source* are blog posts, model cards or commentary.

1. Overview and results (slides 1-5b)

- **Microsoft AI (2026).** *Frontier pre-training at Microsoft AI (MAI-Base-2 report).* `https://microsoft.ai/news/`. The document the whole deck follows: MAI-Base-2's architecture, the scaling ladders, hyperparameter laws, MoE and Muon systems work, clipping, and the data mixture; Figures 1-10 are reproduced from it. Not on arXiv; the PDF was supplied for the lecture.
- **Chowdhery et al. (2022).** *PaLM: scaling language modeling with Pathways*. arXiv:2204.02311. Defines model FLOPs utilization (MFU), the 21% figure the report quotes; Section 5.1 also describes restarting around loss spikes, the practice MAI-Base-2 avoided.
- **Deletang et al. (2023).** *Language modeling is compression*. arXiv:2309.10668. Why a language model's log-loss is a compression rate: the arithmetic-coding argument behind reading bits per byte as a compression score.

2. The model (slides 6-9)

- **Shazeer et al. (2017).** *Outrageously large neural networks: the sparsely-gated mixture-of-experts layer*. arXiv:1701.06538. Sparse top-k gating at scale: a learned router makes a layer's cost independent of its parameter count, and introduces the load-balancing losses later work inherits.
- **Lepikhin et al. (2020).** *GShard: scaling giant models with conditional computation and automatic sharding*. arXiv:2006.16668. Top-2 MoE in transformers at 600B parameters; expert parallelism, capacity factors and token dropping as used since.
- **Fedus, Zoph and Shazeer (2021).** *Switch Transformers: scaling to trillion parameter models with simple and efficient sparsity*. arXiv:2101.03961. Top-1 routing, the auxiliary load-balancing loss with its 0.01 weight, and the first trillion-parameter sparse model.
- **Dai et al. (2024).** *DeepSeekMoE: towards ultimate expert specialization in mixture-of-experts language models*. arXiv:2401.06066. Fine-grained expert segmentation and shared experts, the design MAI-Base-2's 1 shared + 7 of 512 layout follows.
- **Ainslie et al. (2023).** *GQA: training generalized multi-query transformer models from multi-head checkpoints*. arXiv:2305.13245. Grouped-query attention: query heads share key/value heads, shrinking the KV cache by the group size; MAI-Base-2 uses 48 queries per KV head in global layers and 6 in local ones.
- **Press and Wolf (2017).** *Using the output embedding to improve language models*. arXiv:1608.05859. Tied input and output embeddings, the default MAI-Base-1 used and MAI-Base-2 abandoned.

3. Scaling ladders (slides 10-14)

- **Kaplan et al. (2020).** *Scaling laws for neural language models*. arXiv:2001.08361. Power laws in parameters, data and compute over seven orders of magnitude, and the 6ND cost model behind every FLOPs number in the deck.

- **Hoffmann et al. (2022).** *Training compute-optimal large language models*. arXiv:2203.15556. Chinchilla: the parametric fit L = E + A/N^a + B/D^b and the roughly 20 tokens-per-parameter optimum that MAI-Base-2's 625 TPP sits far beyond.
- **Porian et al. (2024).** *Resolving discrepancies in compute-optimal scaling of language models*. arXiv:2406.19146. Why Kaplan and Chinchilla disagreed: embedding parameters, warmup and learning-rate tuning at small scale; a warning about what a ladder must control.
- **Muennighoff et al. (2023).** *Scaling data-constrained language models*. arXiv:2305.16264. Repeating data up to about four epochs is nearly free, then returns decay; the regime that source scaling reproduces on every rung.
- **Bhagia et al. (2024).** *Establishing task scaling laws via compute-efficient model ladders*. arXiv:2412.04403. The ladder method made explicit: a family of small models with one recipe predicts the large model's task performance within a few points.
- **Choshen et al. (2024).** *A hitchhiker's guide to scaling law estimation*. arXiv:2410.11840. How to fit and extrapolate laws in practice: how many models, which sizes, intermediate checkpoints, and the usual errors.

4. Hyperparameter laws (slides 15-18)

- **Loshchilov and Hutter (2019).** *Decoupled weight decay regularization*. arXiv:1711.05101. AdamW: weight decay applied as a multiplicative shrink (1 - lr*wd) rather than as an L2 term in the gradient, which is why the product lr*wd is the quantity that matters.
- **Wang and Aitchison (2024).** *How to set AdamW's weight decay as you scale model and dataset size*. arXiv:2405.13698. The EMA view of AdamW with timescale 1/(lr*wd), and the argument that the timescale should be a fixed fraction of training; the report's exponent of -1 is this rule.
- **Malladi et al. (2022).** *On the SDEs and scaling rules for adaptive gradient algorithms*. arXiv:2205.10287. Derives the square-root scaling rule for Adam-type optimizers: double the batch, multiply the learning rate by sqrt(2); the report's exponent of -0.5 in steps.
- **Yang et al. (2022).** *Tensor Programs V: tuning large neural networks via zero-shot hyperparameter transfer*. arXiv:2203.03466. muP: parameterize so the optimal learning rate does not move with width; the alternative to fitting empirical hyperparameter laws.
- **DeepSeek-AI (2024).** *DeepSeek LLM: scaling open-source language models with longtermism*. arXiv:2401.02954. Empirical scaling laws for learning rate and batch size against compute, fitted the same way as the report's laws for LR and weight decay.
- **Hu et al. (2024).** *MiniCPM: unveiling the potential of small language models with scalable training strategies*. arXiv:2404.06395. Warmup-stable-decay schedules and muP in a practical small-model ladder; how hyperparameter sweeps are made affordable.
- **Srivastava et al. (2014).** *Dropout: a simple way to prevent neural networks from overfitting*. `https://jmlr.org/papers/v15/srivastava14a.html`. The regularizer whose rate the report sets by a scaling law in active parameters. JMLR 15. Not on arXiv.

5. MoE systems (slides 19-23)

- **Gale et al. (2022).** *MegaBlocks: efficient sparse training with mixture-of-experts*. arXiv:2211.15841. Dropless MoE with block-sparse kernels: no token dropping, but memory that moves with the routing; the design MAI's fixed-memory multi-round scheme improves on.
- **Rajbhandari et al. (2022).** *DeepSpeed-MoE: advancing mixture-of-experts inference and training to power next-generation AI scale*. arXiv:2201.05596. Expert parallelism at scale in a production training stack, with the communication and memory bookkeeping that per-expert overhead comes from.
- **Hwang et al. (2022).** *Tutel: adaptive mixture-of-experts at scale*. arXiv:2206.03382. Adaptive parallelism and pipelining for MoE layers, including dynamic capacity factors; the systems background for dispatch/collect scheduling.
- **Zoph et al. (2022).** *ST-MoE: designing stable and transferable sparse expert models*. arXiv:2202.08906. The router z-loss and a careful study of MoE training stability; how capacity factor and token dropping affect quality.
- **Wang et al. (2024).** *Auxiliary-loss-free load balancing strategy for mixture-of-experts*. arXiv:2408.15664. Balancing expert load with a per-expert bias instead of a loss term; the alternative to the batched load-balancing loss the report describes.
- **Korthikanti et al. (2022).** *Reducing activation recomputation in large transformer models*. arXiv:2205.05198. Activation recomputation (rematerialization) and how to minimize it; the compute that memory freed from MoE buffers buys back.

6. Muon (slides 24-25)

- **Jordan (2024).** *Muon: an optimizer for hidden layers in neural networks*. `https://kellerjordan.github.io/posts/muon/`. The origin of Muon: orthogonalize the momentum of each weight matrix with five bfloat16 Newton-Schulz iterations. Blog post; not on arXiv.
- **Bernstein and Newhouse (2024).** *Old optimizer, new norm: an anthology*. arXiv:2409.20325. Muon as steepest descent under the spectral norm; why equalizing singular values of the update is the natural step for a matrix parameter.
- **Liu et al. (2025).** *Muon is scalable for LLM training*. arXiv:2502.16982. Weight decay and update-scale adjustments that make Muon work at scale; about twice AdamW's compute efficiency on a 16B MoE.
- **Kimi Team (2025).** *Kimi K2: open agentic intelligence*. arXiv:2507.20534. A 1T-parameter MoE trained with MuonClip over 15.5T tokens with no loss spikes; the largest public Muon run before MAI-Base-2.
- **Rajbhandari et al. (2020).** *ZeRO: memory optimizations toward training trillion parameter models*. arXiv:1910.02054. Sharding optimizer state across data-parallel GPUs, which is why Muon's momentum has to be resharded before Newton-Schulz.
- **Wen et al. (2025).** *Fantastic pretraining optimizers and where to find them*. arXiv:2509.02046. A controlled comparison of Muon, SOAP, Shampoo and AdamW across scales; how much of the gain survives careful tuning.

7. Numerics and stability (slides 26-29)

- **Micikevicius et al. (2022).** *FP8 formats for deep learning*. arXiv:2209.05433. E4M3 and E5M2: the two 8-bit floats, their ranges, and tensor-wise scaling with a delayed scale, MAI-Base-1's scheme.

- **Rouhani et al. (2023).** *Microscaling data formats for deep learning*. arXiv:2310.10537. MX formats: blocks of 32 values sharing an 8-bit power-of-two scale; MXFP8 is MAI-Base-2's training format.
- **NVIDIA (2025).** *Recipes for pre-training LLMs with MXFP8*. arXiv:2506.08027. How to train with MXFP8 in practice: scale rounding, which tensors to quantize, and matching BF16 on an 8B model over 15T tokens.
- **Fishman et al. (2024).** *Scaling FP8 training to trillion-token LLMs*. arXiv:2409.12517. FP8 instabilities that appear only after hundreds of billions of tokens, and their fix; why a stable FP8 recipe must be tested at full horizon.
- **Pascanu, Mikolov and Bengio (2013).** *On the difficulty of training recurrent neural networks*. arXiv:1211.5063. Global-norm gradient clipping, the standard guard that MAI-Base-1 used alone and MAI-Base-2 supplements with activation gradient clipping.
- **Dehghani et al. (2023).** *Scaling vision transformers to 22 billion parameters*. arXiv:2302.05442. Attention-logit growth as a cause of divergence and QK-normalization as the fix; the canonical example of a stability fix found by looking inside the network.
- **Wortsman et al. (2023).** *Small-scale proxies for large-scale transformer training instabilities*. arXiv:2309.14322. Reproduces loss spikes at small scale by raising the learning rate; QK-norm, z-loss and the LR sensitivity of large models; what a ladder can and cannot reveal about stability.
- **OLMo Team (2024).** *2 OLMo 2 furious*. arXiv:2501.00656. A fully open run that documents its stability fixes (initialization, normalization, z-loss) and its gradient-norm traces; a public counterpart to Figures 5 and 7.

8. Data and vision (slides 30-32)

- **Penedo et al. (2024).** *The FineWeb datasets: decanting the web for the finest text data at scale*. arXiv:2406.17557. A documented web-data pipeline (parse, filter, dedup, classify) with ablations for each stage; the open counterpart to MAI's in-house pipeline.
- **Li et al. (2024).** *DataComp-LM: in search of the next generation of training sets for language models*. arXiv:2406.11794. A benchmark for data curation at fixed compute; quality classifiers give the largest gains, as the report also finds.
- **Abbas et al. (2023).** *SemDeDup: data-efficient learning at web-scale through semantic deduplication*. arXiv:2303.09540. Embedding-based removal of near-duplicates that exact matching misses; the technique behind diversity sampling.
- **Tirumala et al. (2023).** *D4: improving LLM pretraining via document de-duplication and diversification*. arXiv:2308.12284. Picks a diverse subset in embedding space and beats random selection at equal tokens; the closest published analogue of the report's diversity sampling.
- **Ye et al. (2024).** *Data mixing laws: optimizing data mixtures by predicting language modeling performance*. arXiv:2403.16952. Loss as a function of domain proportions, fitted on small runs to choose the mix for a large one; how a Figure 10 mixture can be chosen rather than guessed.
- **Dosovitskiy et al. (2020).** *An image is worth 16x16 words: transformers for image recognition at scale*. arXiv:2010.11929. Images as sequences of linearly embedded patches; the patch embedding that an encoder-free VLM feeds straight into the language model.

- **Bavishi et al. (2023).** *Fuyu-8B: a multimodal architecture for AI agents*. https://www.adept.ai/blog/fuyu-8b. The first widely used encoder-free VLM: image patches projected linearly into a decoder-only transformer, any resolution. Blog post; not on arXiv.
- **Diao et al. (2024).** *Unveiling encoder-free vision-language models*. arXiv:2406.11832. A systematic study of removing the vision encoder: what data and training it takes for a pure decoder to match encoder-based VLMs.
- **Beyer et al. (2024).** *PaliGemma: a versatile 3B VLM for transfer*. arXiv:2407.07726. The encoder-plus-projector design the slide contrasts with MAI's, done carefully; useful for seeing exactly what the encoder-free route gives up and gains.

9. Comparators and frontier reports (slides 5b and 33)

- **DeepSeek-AI (2024).** *DeepSeek-V3 technical report*. arXiv:2412.19437. The reference open frontier recipe: 671B/37B MoE with 1 shared + 8 of 256 experts, block-scaled FP8 training, and multi-token prediction; the design MAI-Base-2's MoE layout is closest to.
- **Qwen Team (2025).** *Qwen3 technical report*. arXiv:2505.09388. The lineage of the Qwen3.8 comparator: fine-grained MoE, GQA, and a documented pre-training mixture of 36T tokens.
- **Yang et al. (2024).** *Gated Delta Networks: improving Mamba2 with delta rule*. arXiv:2412.06464. The linear-attention layer (Gated DeltaNet) that Qwen3.8 interleaves 3:1 with full attention, per its model card.
- **Kimi Team (2025).** *Kimi Linear: an expressive, efficient attention architecture*. arXiv:2510.26692. Kimi Delta Attention, the linear layer Kimi K3 mixes with gated MLA, per its model card.
- **Meta (2025).** *The Llama 4 herd: the beginning of a new era of natively multimodal AI innovation*. https://ai.meta.com/blog/llama-4-multimodal-intelligence/. Natively multimodal MoE models with interleaved local and global attention (iRoPE); a contemporary of the MAI design. Blog post; not on arXiv.
- **DeepSeek-AI (2026).** *DeepSeek-V4-Pro-Base model card (Hugging Face)*. https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro-Base. Source of the comparator row on slide 5b: 1.6T total / 49B active, 61 layers, compressed sparse attention, base weights trained on more than 32T tokens. Model card; secondary source.
- **Moonshot AI (2026).** *Kimi-K3 model card (Hugging Face)*. https://huggingface.co/moonshotai/Kimi-K3. Source of the comparator row on slide 5b: 2.8T / 104B, 93 layers, 896 experts with 16 active and 2 shared, 69 Kimi Delta Attention + 24 gated MLA layers, MXFP4 weights. Model card; secondary source.
- **Qwen Team (2026).** *Qwen3.8-2.4T-A95B model card (Hugging Face)*. https://huggingface.co/Qwen/Qwen3.8-2.4T-A95B. Source of the comparator row on slide 5b: 2.4T / 95B, 92 layers, 512 experts with 10 active and 1 shared, 69 Gated DeltaNet + 23 attention layers. Model card; secondary source.
- **Xiaomi (2026).** *MiMo-V2.6-Pro-RL model card (Hugging Face)*. https://huggingface.co/XiaomiMiMo/MiMo-V2.6-Pro-RL. Source of the comparator row on slide 5b: 1.02T / 42B, 70 layers, 384 experts with 8 active, sliding-window plus global attention. Model card; secondary source.

Index of terms

activation gradient, 38
active fraction, 12
active parameters (N_{active}), 5
all-gather, 35
all-reduce, 32
all-to-all, 29
arithmetic coding, 7
arXiv identifier, 47
aspect ratio, 16

base model, 4
baseline vs candidate, 27
bits per byte (BPB), 7
block-sparse GEMM, 29

capacity factor (CF), 29
checkpoint, 25
clip event, 41
clipping threshold (c), 38
co-design loop, 27
collective, 28
comparator, 9
compute stream, communication stream, 31
compute-optimal, 17
context length, 46

data parallelism, 35
data-constrained scaling, 18
decade, 39
decontamination, 7
decoupled weight decay, 22
deduplication, 42
delayed scaling, 37
dispatch, collect, 29
distillation, 42
diversity sampling, 15
dropless MoE, 29
dropout, 24

E4M3, E5M2, E8M0, 36
effective learning rate, 39
efficiency gain (EG), 15, 27
efficiency gain by rung, 41
EMA timescale, 22
empirical minimum, 23
encoder-free VLM, 45
epoch, pass, 18
equal-quality compute, 20
expert FFN dimension, 11
expert group (EG), 31
expert parallelism, 13
expert parallelism (EP), 29
exploding gradients, 38
exposed communication, 31
extrapolation, 16

final checkpoint, 19
fine-grained experts, 10
first principles, 46
fit minimum, 23
fit weight, 23
fixed recipe, 16
FLOPs, 6
FLOPs-efficiency, 20
Foundry, 42
FP8, 36
frontier model, 4

GB200, 6
GEMM, 28
global norm, 38
goodput, 6
gradient clipping, 38
gradient norm, 39
grouped GEMM, 32
grouped-query attention (GQA), 5

HBM, 32
held-out evaluation, 7
hidden dimension (d), 11
host–device synchronization, 32
hyperparameter scaling law, 24
hyperparameter sweep, 22

interleaved documents, 45
irreducible loss (E), 17
iteration (step) time, 35

kernel, 27
kernel launch overhead, 28
knowledge cutoff, 42
KV cache, 14

ladder fit, 19
LBL stats, 31
learning rate (LR, η), 22
learning-rate decay, 25
linear attention, Gated DeltaNet, Kimi Delta Attention, 10
load balancing, 13
load imbalance, 28
load-balancing loss, 32
local and global layers, 5
loss spike, 25

malformed example, 41
memory fragmentation, 28
microscaling (MX), 37
mixture, 43
mixture of experts (MoE), 4
model card, 10
model FLOPs utilization (MFU), 6
model selection, 24
model–infrastructure co-design, 15
MoE down projection, 11
MoE-layer MFU, 28
momentum, 33
multi-head latent attention (MLA), 10
multi-query attention (MQA), 14
Muon, 33
MuonClip, 34
MXFP4, 10
MXFP8, 12
MXFP8, MXFP4, 37

nat, bit, 7
native multimodal pre-training, 45
negative log-likelihood (NLL), 7
Newton–Schulz iteration, 34
normalization placement, 46
NVLink, 32

open-weight model, 9
organic data, 42
orthogonalization, 34
outlier, 37
outlier example, 38
over-training, 17
overflow, saturation, 37

patch, 45
PDFs, books, STEM, 43
per-example gradient, 38
per-step trace, 39
positional encoding, 46
post-trained checkpoint, 7
power law, 17
power law with a floor, 19
pre-training, 4
predicted vs actual final, 19
prefetching, 35
preprocessing pipeline, 41
projection onto a scaling law, 20
projector, 45
provenance, 42

QK-normalization, 25
quadratic surface fit, 23
quality classifier, 42
query, key, value heads, 14

recipe, 5
reference model, 20
regularization, 18
rematerialization, 6
repeated tokens, 43
resharding, 35
robots.txt, 42
rollback, 25
rolling median, 39
round, 31
routed vs shared expert, 13
router (gate), 13
rung, 16
runner-up margin, 9

scale factor, 37
scale-dependent gain, 41
scaling ladder, 4
scaling law, 17
secondary source, 47
SemDeDup, D4, 42
shard, 35
shared expert, 10
sigmoid, 24
singular value decomposition (SVD), 33
sliding window, 14
sliding-window attention, 10
source scaling, 15
sparsely gated MoE, 13
spectral norm, 34

square-root scaling rule, 24
steepest descent under a norm, 34
straggler, 35
symmetric memory, 32

tail effect, 32
the 6*ND* rule, 6
tied / untied embeddings, 11
token capacity, 31
token dropping, 29
tokens per parameter (TPP), 16
top-k routing, 13
topic binning, 42
total parameters (N_{total}), 5
training loss, 25
training steps, 24

under-regularized, 18
unembedding, 14
unique tokens, 18

vision encoder, 44
vision-language model (VLM), 44
vision-only data, 43
visual grounding, 45
VLM corpus, 43
vocabulary size (V), 46

wall-clock time, 27
weight decay (WD, λ), 22
weight quantization (Weight Quant), 31
weight tying, 14
weighted evaluation NLL, 23

z-loss, 25
ZeRO, 35